

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA DE SOFTWARE

ASIGNATURA: APLICACIONES WEB

CURSO: 5TO SOFTWARE “A”

**Sistema de Gestión de Recursos Operativos,
Administrativos y de Seguridad para
Cooperativas de Transporte (SGROAS)**

Informe del Proyecto Fin de Curso — Versión 1.0.0 — Entrega Final

<https://github.com/Alxjandr07/SGROAS-ProyectoAppWeb.git>

URL pública: <https://sgroas-backend.onrender.com>

Commit: 9ded2a69 | DOI Software: 10.5281/zenodo.22522109 | DOI Dataset:
10.5281/zenodo.21973297

DOCENTE:

ING. GUERRERO ULLOA
GLEISTON CICERÓN

ESTUDIANTES:

ESCUDERO PLAZA MARÍA DEL
ROSARIO

ORCID: 0009-0007-3212-9924

TEJADA BAJAÑA LUIS ALEJANDRO

ORCID: 0009-0006-2080-1798

PERÍODO ACADÉMICO:

2026–2027 PPA

FECHA:

01 de septiembre de 2026

Quevedo — Ecuador

Resumen

Resumen (ES):

El presente informe documenta el diseño, implementación y validación del Sistema de Gestión de Recursos Operativos, Administrativos y de Seguridad para Cooperativas de Transporte (SGROAS), aplicación web de tres capas desarrollada como Proyecto Fin de Curso en la Universidad Técnica Estatal de Quevedo. SGROAS gestiona recursos operativos, administrativos y de seguridad mediante arquitectura basada en Angular 17+ (frontend), Spring Boot 3.2.x con Java 21 LTS (backend), PostgreSQL 16 y Redis 7. Ofrece autenticación stateless mediante JWT en cookie HttpOnly, CRUD completo para conductores, usuarios, rutas, vehículos, asignaciones e incidentes, seis procedimientos almacenados/funciones SQL y reportes. La verificación se realizó mediante 150 pruebas JUnit 5 con cobertura JaCoCo del 85,5 % líneas / 54,3 % ramas en el núcleo (67,5 % líneas full con `abd` excluido del `jacoco:check`), pruebas de carga k6 (3 corridas, 50 VUs, 30 s) con media de medias 23,01 ms IC95 % [-7,88; 53,91], p95 máximo 173,13 ms (umbral 200 ms, 0 % errores) y análisis de seguridad OWASP con ZAP baseline. Los requisitos se especificaron conforme a ISO/IEC/IEEE 29148:2018 y se verificaron las 15 características del INCOSE v4. El SUS obtuvo media 63,0 (DT 13,88, IC95 % [53,07; 72,93], n=10). El sistema se versiona como v1.0.0 con DOI Zenodo.

Palabras clave: ingeniería de requisitos, SRS, Spring Boot, Angular, JWT, PostgreSQL, OWASP, ISO 29148, INCOSE, proyecto fin de curso.

Abstract (EN):

This report documents the design, implementation and validation of the

Operations, Administrative and Security Resource Management System for Transport Cooperatives (SGROAS), a three-tier web application developed as Final Course Project at the State Technical University of Quevedo. SGROAS manages operational, administrative and security resources through an architecture based on Angular 17+ (frontend), Spring Boot 3.2.x with Java 21 LTS (backend), PostgreSQL 16 and Redis 7. It provides stateless JWT authentication in HttpOnly cookies, full CRUD for drivers, users, routes, vehicles, assignments and incidents, six stored procedures/SQL functions and reports. Verification included 150 JUnit 5 tests with JaCoCo 85.5 % lines / 54.3 % branches on the core (67.5 % lines full, `abd` excluded from `jacoco:check`), k6 load tests (3 runs, 50 VUs, 30 s) with mean-of-means 23.01 ms 95 %CI [-7.88; 53.91], max p95 173.13 ms (threshold 200 ms, 0 % errors) and OWASP ZAP baseline scan. Requirements follow ISO/IEC/IEEE 29148:2018 and all 15 INCOSE v4 characteristics were verified. SUS mean was 63.0 (SD 13.88, 95 %CI [53.07; 72.93], n=10). The system is versioned as v1.0.0 with Zenodo DOI.

Keywords: requirements engineering, SRS, Spring Boot, Angular, JWT, PostgreSQL, OWASP, ISO 29148, INCOSE, final course project.

Índice general

Resumen	2
Lista de Siglas y Abreviaturas	13
1. Introducción	15
1.1. Contexto y justificación	15
1.2. Descripción del problema	16
1.3. Preguntas de investigación	17
1.4. Objetivos	17
1.4.1. Objetivo general	17
1.4.2. Objetivos específicos	18
1.5. Contribuciones	18
1.6. Estructura del documento	19
2. Marco Teórico	21
2.1. Ingeniería de requisitos	21
2.1.1. ISO/IEC/IEEE 29148:2018	21
2.1.2. INCOSE Guide to Writing Requirements v4	22
2.1.3. Historias de usuario y casos de uso	23
2.1.4. Matriz MoSCoW	24

2.2.	Arquitectura de software	24
2.2.1.	Modelo C4	24
2.2.2.	Arquitectura de tres capas	25
2.3.	Seguridad	25
2.3.1.	OWASP Top 10:2021	25
2.3.2.	JWT (RFC 7519)	26
2.3.3.	ProblemDetails (RFC 7807)	26
2.4.	Tecnologías de implementación	27
2.4.1.	Spring Boot 3.2.x con Java 21 LTS	27
2.4.2.	Angular 20	27
2.4.3.	PostgreSQL 16	27
2.4.4.	Redis 7	28
2.5.	Estándares de calidad	28
2.5.1.	ISO/IEC 25010:2011	28
2.5.2.	Métricas de calidad de código	29
3.	Trabajos Relacionados	30
3.1.	Estrategia de búsqueda	30
3.1.1.	Bases de datos consultadas	30
3.1.2.	Cadena de búsqueda	30
3.1.3.	Criterios de inclusión y exclusión	31
3.2.	Diagrama de flujo PRISMA	31
3.3.	Síntesis de trabajos incluidos	32
3.4.	Brecha identificada	35
4.	Ingeniería de Requisitos	36

4.1. Stakeholders	36
4.2. Proceso de elicitación	37
4.3. Historias de usuario	38
4.4. Casos de uso	39
4.5. Priorización MoSCoW	40
4.6. Matriz de trazabilidad	41
4.7. Conformidad INCOSE	42
4.8. Gestión del cambio	42
4.9. Resumen de requisitos	43
5. Materiales y Métodos	44
5.1. Enfoque metodológico global	44
5.2. Goal-Question-Metric (GQM)	45
5.3. Instrumentos de medición	46
5.4. Población, muestreo y participantes	47
5.4.1. Sistema (rendimiento, seguridad, cobertura, calidad web)	47
5.4.2. Participantes SUS (usabilidad)	47
5.5. Protocolo experimental	47
5.5.1. Rendimiento (k6)	47
5.5.2. Seguridad (OWASP + ZAP)	48
5.5.3. Usabilidad (SUS)	48
5.5.4. Cobertura (JaCoCo)	48
5.5.5. Calidad web (Lighthouse)	49
5.6. Análisis estadístico	49
5.7. Semillas y determinismo	49

5.8. Protocolo de reproducibilidad	50
6. Diseño del Sistema y Arquitectura	51
6.1. Arquitectura general	51
6.1.1. Contexto (C4 nivel 1)	51
6.1.2. Contenedores (C4 nivel 2)	51
6.1.3. Componentes (C4 nivel 3)	52
6.2. Decisiones de Arquitectura (ADR)	52
6.2.1. ADR-001: Arquitectura monolítica con Spring Boot	53
6.2.2. ADR-002: JPA + Hibernate con Flyway para persistencia	53
6.2.3. ADR-003: Autenticación stateless con JWT	53
6.2.4. ADR-004: Cache distribuido con Redis	53
6.2.5. ADR-005: Diseño de base de datos con soft delete	53
6.2.6. ADR-006: Estrategia híbrida de acceso a datos (CRUD-ORM + SP)	54
6.2.7. ADR-007: Despliegue público en Render (PaaS)	54
6.3. Diseño de la base de datos	55
6.4. Diseño de la API REST	55
6.5. Matriz de trazabilidad	57
7. Implementación	58
7.1. Configuración general	58
7.2. Aislamiento de capas	58
7.3. Manejo de errores globales	59
7.4. Esquema de caching	59
7.5. Mecanismo de seguridad	59

7.6. Estrategia híbrida de acceso a datos	60
7.6.1. CRUD con Spring Data JPA	60
7.6.2. Stored procedures con @NamedStoredProcedureQuery	61
7.6.3. Funciones SQL	62
7.7. Estrategia de seguridad estática	62
7.8. Pipeline de CI	63
8. Evaluación	64
8.1. Rendimiento (Bloque C.1) — RQ1	64
8.2. Seguridad (Bloque C.2) — RQ2	65
8.3. Usabilidad (Bloque C.3) — RQ3	66
8.4. Cobertura de pruebas (Bloque C.4)	67
8.5. Calidad web (Bloque C.5) — RQ4	67
8.6. Síntesis	68
9. Discusión	69
9.1. Respuesta a las preguntas de investigación	69
9.2. Comparación con trabajos relacionados	70
9.3. Hallazgos inesperados	70
9.4. Implicaciones prácticas	71
10. Amenazas a la validez	72
10.1. Amenazas a la validez de constructo	72
10.2. Amenazas a la validez interna	73
10.3. Amenazas a la validez externa	73
10.4. Amenazas a la validez de conclusión	74

10.5. Conclusión	74
11.Trabajo futuro	75
12.Conclusiones	76
Declaraciones	84
A. Cadena de búsqueda de trabajos relacionados	87
B. Protocolo SUS	88
C. Configuración de integración continua	89
D. Checklists de verificación	91
E. Observaciones de auditoría	92
Anexo A: Matriz de resultados por bloque	94

Índice de figuras

C.1. Pipeline CI (GitHub Actions) con tres ejecuciones consecutivas exitosas (criterio P1). Commits <code>4f4e6cd</code> , <code>17f484a</code> , <code>66fa254</code> en <code>main</code> . . .	89
C.2. Despliegue en Render con estado <code>Deploy succeeded Live (3m14s)</code> y arranque verificado (<code>Tomcat started on port 8080 Started SgroasApplication</code>).	90
C.3. Servicio <code>sgroas-backend</code> en estado <code>Live</code> con URL pública <code>https://sgroas-backend.onrender.com</code> (Blueprint managed).	90

Índice de cuadros

3.1. Trabajos relacionados y comparación con SGROAS.	32
4.4. Distribución de requisitos por prioridad MoSCoW	41
4.5. Resumen de conformidad INCOSE C1–C15	42
5.1. Desglose Goal-Question-Metric del estudio empírico.	45
6.1. Endpoints principales de la API REST.	56
8.1. Síntesis de resultados por bloque de evaluación.	68
E.2. Resumen de mediciones por bloque.	94

Listings

7.1. Repositorio de conductores con consultas derivadas.	60
7.2. Declaración @NamedStoredProcedureQuery en Incidente.	61
7.3. Invocación del SP desde el repositorio.	61
7.4. Invocación de función SQL escalar.	62

Lista de Siglas y Abreviaturas

Sigla	Significado
ABD	Aplicación Basada en Datos
ADR	Architecture Decision Record
API	Application Programming Interface
BCrypt	Blowfish Cipher (algoritmo de hash)
C4	Model of Architectural Decomposition (4 niveles)
CI	Continuous Integration
CRUD	Create, Read, Update, Delete
DSR	Design Science Research
DTO	Data Transfer Object
FAIR	Findable, Accessible, Interoperable, Reusable
GQM	Goal-Question-Metric
HTTPS	Hypertext Transfer Protocol Secure
IC	Intervalo de Confianza
INCOSE	International Council on Systems Engineering
ISO	International Organization for Standardization
JaCoCo	Java Code Coverage
JWT	JSON Web Token
JWT	JSON Web Token (RFC 7519)
Lighthouse	Auditor de calidad web (Google)
NFR	Non-Functional Requirement
OWASP	Open Web Application Project Security

Sigla	Significado
OWASP	Open Worldwide Application Security Project
PaaS	Platform as a Service
PFC	Proyecto Fin de Curso
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
REST	Representational State Transfer
SRS	Software Requirements Specification
SP	Stored Procedure
SUS	System Usability Scale
UTEQ	Universidad Técnica Estatal de Quevedo
VU	Virtual User (k6)
ZAP	Zed Attack Proxy (OWASP)

Capítulo 1

Introducción

1.1. Contexto y justificación

Las cooperativas de transporte urbano en Ecuador enfrentan desafíos significativos en la gestión de sus recursos operativos, administrativos y de seguridad. La planificación de rutas, la asignación de conductores y vehículos, el registro de incidentes y la generación de reportes se realizan frecuentemente mediante procesos manuales o semi-automatizados que dependen de hojas de cálculo, formatos en papel y sistemas de comunicación no estructurados [1]. Estas prácticas generan inconsistencias en los datos, demoras en la toma de decisiones y dificultades para el seguimiento del cumplimiento normativo.

El sector del transporte público en Ecuador mueve a más de 3 millones de pasajeros diarios en las principales ciudades, según datos del Ministerio de Transporte y Obras Públicas [2]. La gestión ineficiente de la flota, las asignaciones de rutas sin herramientas digitales y la falta de un registro estructurado de incidentes de seguridad representan riesgos operativos que afectan directamente la calidad del servicio y la seguridad de los usuarios.

En este contexto, el desarrollo de un sistema de información web que centralice la gestión de los recursos de una cooperativa de transporte se convierte en una necesidad estratégica. SGROAS (Sistema de Gestión de Recursos Operativos,

Administrativos y de Seguridad) fue concebido como una solución que integra las funcionalidades esenciales para la administración de conductores, usuarios, rutas, vehículos, asignaciones operativas e incidentes, proporcionando una plataforma unificada para el personal administrativo, operativo y de seguridad de la institución.

La motivación del proyecto se sustenta en tres pilares: (1) la necesidad real expresada por el personal de la cooperativa durante las entrevistas iniciales, (2) la oportunidad de aplicar los conocimientos adquiridos durante la carrera de Ingeniería de Software en un caso de uso real, y (3) la contribución al cuerpo de conocimiento en ingeniería de requisitos aplicada a sistemas de gestión de transporte en el contexto ecuatoriano.

1.2. Descripción del problema

La cooperativa de transporte objeto de estudio carece de un sistema integrado de información que soporte sus procesos operativos. Actualmente, la gestión de conductores se realiza mediante registros en hojas de cálculo que no garantizan la integridad de los datos. La asignación de rutas y vehículos se coordina por teléfono o mensajería instantánea, lo que dificulta el seguimiento y la auditoría. El registro de incidentes se realiza en formatos físicos que pueden extraviarse o ser incompletos. La generación de reportes operativos requiere la consolidación manual de datos de múltiples fuentes, proceso que consume tiempo y es propenso a errores.

Estos problemas se traducen en:

- **Pérdida de información:** Registros en papel y hojas de cálculo no respaldadas generan pérdida de datos históricos.
- **Ineficiencia operativa:** La asignación manual de recursos consume tiempo valioso del personal coordinador.
- **Falta de trazabilidad:** No existe un registro estructurado de las decisiones operativas y los incidentes.

- **Limitaciones en reportes:** La ausencia de herramientas automatizadas dificulta la obtención de métricas para la toma de decisiones.
- **Riesgos de seguridad:** La gestión descentralizada de la información aumenta la exposición a incidentes de seguridad de datos.

1.3. Preguntas de investigación

El desarrollo de SGROAS se orienta a responder las siguientes preguntas de investigación:

1. **RQ1:** ¿Cómo se pueden especificar los requisitos de un sistema de gestión de recursos de transporte utilizando estándares internacionales (ISO/IEC/IEEE 29148:2018) y verificando las características de calidad del INCOSE Guide to Writing Requirements v4?
2. **RQ2:** ¿Es factible implementar una arquitectura de tres capas (Angular + Spring Boot + PostgreSQL) que cumpla con los requisitos funcionales y no funcionales definidos para el sistema SGROAS?
3. **RQ3:** ¿Qué nivel de calidad de código y rendimiento se puede alcanzar aplicando buenas prácticas de ingeniería de software en un proyecto de estas características?
4. **RQ4:** ¿Cómo afecta la usabilidad percibida (SUS) de una aplicación de gestión de transporte a la satisfacción de sus usuarios potenciales?

1.4. Objetivos

1.4.1. Objetivo general

Diseñar, implementar y validar el Sistema de Gestión de Recursos Operativos, Administrativos y de Seguridad (SGROAS), una aplicación web de tres capas

que gestione los recursos de una cooperativa de transporte, verificando la calidad de sus requisitos conforme a ISO/IEC/IEEE 29148:2018 y las 15 características del INCOSE Guide to Writing Requirements v4.

1.4.2. Objetivos específicos

1. Especificar los requisitos funcionales y no funcionales del sistema SGROAS en un documento SRS conforme a ISO/IEC/IEEE 29148:2018, con trazabilidad completa hacia historias de usuario y casos de uso.
2. Verificar las 15 características de calidad de requisitos (C1–C15) del INCOSE Guide to Writing Requirements v4 para cada requisito especificado.
3. Implementar el backend del sistema utilizando Spring Boot 3.2.x con Java 21 LTS, PostgreSQL 16 y Redis 7, aplicando principios de arquitectura limpia y seguridad OWASP.
4. Implementar el frontend del sistema utilizando Angular 20, con autenticación JWT en cookie HttpOnly y diseño responsivo.
5. Ejecutar un conjunto de pruebas automatizadas que incluya pruebas unitarias, de integración y de carga, verificando cobertura mínima del 70 % y p95 menor a 200 ms.
6. Realizar un escaneo de seguridad con OWASP ZAP y verificar el cumplimiento de las cabeceras HTTP de seguridad.
7. Aplicar la prueba de System Usability Scale (SUS) con un mínimo de 15 participantes para evaluar la usabilidad percibida del sistema.

1.5. Contribuciones

Las contribuciones esperadas del proyecto SGROAS son:

1. **Documento SRS v1.0.0:** Especificación completa de requisitos conforme a ISO/IEC/IEEE 29148:2018, con verificación INCOSE C1–C15, 50 requisitos (44 funcionales + 6 no funcionales) y trazabilidad completa.
2. **Sistema funcional:** Aplicación web desplegable con Docker Compose, que integra autenticación JWT, CRUD completo para 6 entidades, 7 procedimientos almacenados y 2 funciones SQL.
3. **Evidencia de calidad:** 150 pruebas JUnit 5 con JaCoCo núcleo 85,5 % líneas / 54,3 % ramas (67,5 % líneas full), escaneo OWASP ZAP baseline y pruebas de carga k6 (3 corridas, media 23,01 ms, p95 máx 173,13 ms).
4. **Validación de requisitos:** Checklist INCOSE verificando las 15 características de calidad para cada requisito del SRS, con evidencia de elicitación documentada.
5. **Dataset reproducible:** Repositorio público en GitHub con código fuente, documentación, scripts de medición y datos semilla, archivado en Zenodo con DOI asignado.

1.6. Estructura del documento

El presente informe se organiza en los siguientes capítulos:

Capítulo 1 — Introducción: Contexto, justificación, descripción del problema, preguntas de investigación, objetivos y contribuciones (el presente capítulo).

Capítulo 2 — Marco Teórico: Fundamentos teóricos y tecnológicos que sustentan el desarrollo del sistema: ISO/IEC/IEEE 29148:2018, INCOSE Guide, modelo C4, JWT, OWASP Top 10, Spring Boot, Angular, PostgreSQL y Redis.

Capítulo 3 — Diseño y Arquitectura: Arquitectura de tres capas, diagramas C4 (niveles 1–3), decisiones de diseño (ADR), diseño de base de datos y diseño de la API REST.

Capítulo 4 — Ingeniería de Requisitos: Proceso de elicitación, stakeholders, historias de usuario, casos de uso, matriz MoSCoW, matriz de trazabilidad y conformidad INCOSE.

Capítulo 5 — Verificación y Validación: Pruebas unitarias e de integración (JaCoCo), pruebas de carga (k6), escaneo de seguridad (OWASP ZAP), prueba de usabilidad (SUS) y observaciones de auditoría.

Capítulo 6 — Conclusiones: Resumen de resultados, respuesta a las preguntas de investigación, limitaciones y trabajo futuro.

Capítulo 2

Marco Teórico

Este capítulo presenta los fundamentos teóricos y tecnológicos que sustentan el diseño e implementación del sistema SGROAS. Se organizan en cuatro áreas principales: ingeniería de requisitos, arquitectura de software, seguridad y tecnologías de implementación.

2.1. Ingeniería de requisitos

2.1.1. ISO/IEC/IEEE 29148:2018

La norma ISO/IEC/IEEE 29148:2018 [3] establece los procesos y productos asociados con la ingeniería de requisitos de software. Define un SRS (Software Requirements Specification) como el documento que captura los requisitos de software de un sistema, incluyendo sus requisitos funcionales, no funcionales y de interfaz.

Las características clave de un SRS conforme a esta norma son:

- **Unambiguous:** Cada requisito se expresa de manera que solo admite una interpretación.
- **Complete:** El SRS incluye todos los requisitos significativos (funcionales, no funcionales y de diseño).

- **Consistent:** Los requisitos no contradictorios entre sí.
- **Verifiable:** Cada requisito puede verificarse mediante una técnica de verificación definida.
- **Traceable:** Cada requisito es rastreable hacia su origen y hacia los elementos de diseño.

SGROAS adopta la estructura de SRS recomendada por ISO/IEC/IEEE 29148:2018, organizando los requisitos en tres categorías: funcionales (REQ-F), no funcionales (REQ-NF) y de interfaz externa (REQ-IF). Cada requisito incluye su identificador, tipo, prioridad MoSCoW, justificación (rationale), enunciado, criterio de aceptación, método de verificación y trazabilidad.

2.1.2. INCOSE Guide to Writing Requirements v4

El INCOSE (International Council on Systems Engineering) publica la guía “Guide to Writing Requirements” [4], que define 15 características de calidad para requisitos individuales y conjuntos de requisitos. Estas características se organizan en dos grupos:

Características individuales (C1–C9):

- C1. **Necessary:** El requisito es necesario para cumplir el objetivo del sistema.
- C2. **Appropriate:** El requisito es apropiado para el nivel de abstracción al que pertenece.
- C3. **Unambiguous:** El requisito se expresa de manera clara y sin ambigüedad.
- C4. **Complete:** El requisito contiene toda la información necesaria.
- C5. **Singular:** El requisito expresa un solo concepto.
- C6. **Feasible:** El requisito es factible de implementar.
- C7. **Verifiable:** El requisito puede verificarse mediante una técnica definida.

C8. **Correct:** El requisito refleja la necesidad real del stakeholder.

C9. **Conforming:** El requisito sigue las convenciones de formato establecidas.

Características del conjunto (C10–C15):

C10. **Complete (set):** El conjunto contiene todos los requisitos necesarios.

C11. **Consistent (set):** Los requisitos no se contradicen entre sí.

C12. **Comprehensible:** El conjunto es comprensible para los stakeholders.

C13. **Able to be validated:** El conjunto puede validarse contra las necesidades del usuario.

C14. **Trazable:** Cada requisito es rastreable hacia su origen.

C15. **Unico (set):** Cada requisito es único dentro del conjunto.

SGROAS verifica las 15 características C1–C15 para cada uno de sus 50 requisitos en el checklist `docs/checklists/incose2023-req.md`.

2.1.3. Historias de usuario y casos de uso

Las historias de usuario, introducidas por Cohn [5], son una técnica de captura de requisitos que se expresa en el formato: “Como [rol], quiero [acción] para [valor]”. Cada historia se valida mediante criterios de aceptación en formato Gherkin (Given/When/Then). Las historias de usuario se evalúan con los criterios INVEST: Independent, Negotiable, Valuable, Estimable, Small y Testable.

Los casos de uso, formalizados por Cockburn [6], describen las interacciones entre actores y el sistema para lograr un objetivo. Se organizan en niveles de detalle (Cockburn 1–4) que van desde el objetivo del usuario hasta el flujo paso a paso. Los casos de uso incluyen escenarios principales de éxito y extensiones (casos de excepción).

SGROAS combina ambas técnicas: 8 historias de usuario con criterios Gherkin (formato Connextra) y 6 casos de uso en formato Cockburn niveles 1–4, garantizando cobertura completa de los requisitos funcionales.

2.1.4. Matriz MoSCoW

La técnica MoSCoW [7] prioriza los requisitos en cuatro categorías:

- **Must:** Requisito obligatorio; el sistema no es útil sin él.
- **Should:** Requisito importante; se incluye si hay recursos disponibles.
- **Could:** Requisito deseable; se incluye si sobra tiempo.
- **Won't:** Requisito pospuesto a futuras versiones.

SGROAS clasifica 44 requisitos como Must (88 %) y 6 como Should (12 %), reflejando un sistema con funcionalidad central consolidada y reportes como capacidades complementarias.

2.2. Arquitectura de software

2.2.1. Modelo C4

El modelo C4 [8] es una técnica de documentación arquitectónica que describe un sistema en cuatro niveles de abstracción:

1. **Contexto (nivel 1):** Describe el sistema en su entorno, mostrando actores y sistemas externos.
2. **Contenedores (nivel 2):** Muestra las aplicaciones y bases de datos que componen el sistema.
3. **Componentes (nivel 3):** Detalla los componentes internos de cada contenedor.

4. **Código (nivel 4):** Describe la estructura de clases o módulos (opcional).

SGROAS documenta su arquitectura en los niveles 1–3 mediante diagramas DSL (Domain Specific Language) generados con Structurizr, ubicados en `docs/arquitectura/`. Los diagramas C4 proporcionan una visión escalonada que facilita la comprensión de la arquitectura para diferentes audiencias.

2.2.2. **Arquitectura de tres capas**

La arquitectura de tres capas separa la presentación (frontend), la lógica de negocio (backend) y el almacenamiento de datos (base de datos) en capas independientes [9]. SGROAS implementa esta arquitectura con:

- **Capa de presentación:** Angular 20, que consume la API REST del backend.
- **Capa de lógica de negocio:** Spring Boot 3.2.x con Java 21 LTS, que implementa los controladores, servicios y repositorios.
- **Capa de datos:** PostgreSQL 16 para persistencia y Redis 7 como caché distribuida.

2.3. **Seguridad**

2.3.1. **OWASP Top 10:2021**

El OWASP (Open Worldwide Application Security Project) publica periódicamente la lista de los 10 riesgos de seguridad más críticos en aplicaciones web [10]. Los riesgos más relevantes para SGROAS son:

A01 — Broken Access Control: El mecanismo de autenticación JWT en cookie `HttpOnly + Secure + SameSite=Strict`, combinado con la autorización por rol, mitiga este riesgo.

A02 — Cryptographic Failures: TLS v1.3 en producción y cifrado de contraseñas con bcrypt mitigan este riesgo.

A03 — Injection: El uso de JPA con consultas parametrizadas y la prohibición de SQL dinámico en procedimientos almacenados mitigan este riesgo.

A05 — Security Misconfiguration: Las cabeceras HTTP de seguridad (HSTS, CSP, X-Frame-Options, X-Content-Type-Options) mitigan este riesgo.

A07 — Identification and Authentication Failures: El rate limiting en login (6 intentos, respuesta 429) mitiga este riesgo.

2.3.2. JWT (RFC 7519)

JSON Web Token (JWT) [11] es un estándar para la representación segura de afirmaciones entre dos partes. Un JWT se compone de tres partes: header (algoritmo y tipo), payload (claims) y signature (firma). SGROAS genera JWTs con 7 claims: `iss` (emisor), `sub` (sujeto), `aud` (audiencia), `exp` (expiración), `nbf` (no antes de), `iat` (emitido en) y `jti` (identificador único). El JWT se almacena en una cookie `HttpOnly + Secure + SameSite=Strict`, evitando el acceso desde JavaScript del navegador.

2.3.3. ProblemDetails (RFC 7807)

El estándar RFC 7807 [12] define un formato uniforme para los errores en APIs HTTP. SGROAS implementa ProblemDetails en todas las respuestas de error, incluyendo los campos `type`, `title`, `status`, `detail` e `instance`. Esto garantiza que los clientes puedan manejar los errores de manera consistente y automatizada.

2.4. Tecnologías de implementación

2.4.1. Spring Boot 3.2.x con Java 21 LTS

Spring Boot [13] es un framework que simplifica la creación de aplicaciones basadas en Spring. SGROAS utiliza Spring Boot 3.2.x con Java 21 LTS (Long Term Support), aprovechando:

- Spring Data JPA con Hibernate para el mapeo objeto-relacional (ORM).
- Spring Security para la autenticación y autorización.
- Spring Cache con Redis para la caché distribuida.
- Flyway para las migraciones de base de datos.
- SpringDoc OpenAPI para la documentación automática de la API.

2.4.2. Angular 20

Angular [14] es un framework de desarrollo frontend mantenido por Google. SGROAS utiliza Angular 20 con TypeScript, aprovechando:

- Componentes y módulos para la organización del código.
- Servicios e inyección de dependencias para la comunicación con la API.
- Angular Router para la navegación entre vistas.
- HttpClient para las peticiones HTTP al backend.

2.4.3. PostgreSQL 16

PostgreSQL [15] es un sistema de gestión de bases de datos relacional de código abierto. SGROAS utiliza PostgreSQL 16 aprovechando:

- Procedimientos almacenados para reportes y agregaciones complejas.

- Funciones SQL para estadísticas y métricas.
- Triggers para auditoría y validación de datos.
- Índices para optimización de consultas.
- Tipos enumerados para estados y categorías.

2.4.4. Redis 7

Redis [16] es un almacén de datos en memoria utilizado como caché distribuida. SGROAS emplea Redis para:

- Caché del listado paginado de conductores con TTL configurable.
- Lista negra de tokens JWT revocados (JTI en logout).
- Rate limiting para el control de intentos de login.

2.5. Estándares de calidad

2.5.1. ISO/IEC 25010:2011

La norma ISO/IEC 25010:2011 [17] define un modelo de calidad del producto software con ocho características: adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad. SGROAS verifica explícitamente las siguientes características:

- **Adecuación funcional:** El sistema cumple con los 44 requisitos funcionales especificados.
- **Eficiencia de desempeño:** El p95 del listado de conductores es de 173,13 ms máx (media de medias 23,01 ms IC95 % [-7,88;53,91], umbral 200 ms, 0 % errores).

- **Seguridad:** Cabeceras HTTP de seguridad, JWT en cookie HttpOnly, rate limiting, parametrización de consultas.
- **Usabilidad:** SUS de 63,0 (DT 13,88, n=10).

2.5.2. Métricas de calidad de código

La calidad de código se mide mediante:

- **Cobertura de pruebas (JaCoCo):** núcleo 85,5 % líneas / 54,3 % ramas (75,8 % instrucciones), full 67,5 % líneas / 30,2 % ramas; abd excluido del `jacoco:check`.
- **Número de pruebas:** 150 pruebas JUnit 5 (unitarias e de integración).
- **Deuda técnica:** Verificación mediante el plugin de SonarQube en CI.

Capítulo 3

Trabajos Relacionados

Este capítulo presenta una revisión estructurada de trabajos previos sobre sistemas de gestión de transporte y arquitecturas web para aplicaciones de gestión de recursos. La búsqueda se documenta siguiendo la orientación metodológica de Kitchenham y Charters [18] y se reporta con la disciplina de PRISMA 2020 [19].

3.1. Estrategia de búsqueda

3.1.1. Bases de datos consultadas

Se consultaron las siguientes bases de datos: IEEE Xplore, ACM Digital Library, Scopus, ScienceDirect y Springer Link. La ventana temporal fue de 2018 a 2026, considerando trabajos de los últimos ocho años para capturar tanto soluciones consolidadas como tendencias recientes.

3.1.2. Cadena de búsqueda

La cadena de búsqueda combina sinónimos y operadores booleanos:

```
("transport management" OR "fleet management" OR "public transit")  
AND ("web application" OR "web system" OR "information system")
```

AND ("Spring Boot" OR "Java" OR "Angular" OR "React" OR "PostgreSQL")
AND ("REST API" OR "microservices" OR "monolith")

3.1.3. Criterios de inclusión y exclusión

Inclusión: (I1) Sistemas web de gestión de transporte o flota; (I2) Utilizan arquitectura de tres capas o similar; (I3) Incluyen evaluación empírica (rendimiento, usabilidad o seguridad); (I4) Publicados en conferencias o revistas del área de ingeniería de software.

Exclusión: (E1) Artículos de opinión o tutoriales sin evaluación; (E2) Sistemas móviles nativos sin componente web; (E3) Trabajos cuyo dominio no es transporte/gestión de flota; (E4) Artículos duplicados entre bases de datos.

3.2. Diagrama de flujo PRISMA

El proceso de selección siguió las cuatro fases de PRISMA 2020 [19]:

1. **Identificación:** 142 registros identificados en las cinco bases de datos (IEEE Xplore: 38, ACM: 29, Scopus: 41, ScienceDirect: 22, Springer: 12).
2. **Cribado:** 96 registros tras eliminación de duplicados (46 duplicados). Se revisaron títulos y resúmenes.
3. **Elegibilidad:** 24 artículos recuperados en texto completo.
4. **Incluidos:** 10 trabajos incluidos en la síntesis final tras aplicar criterios de inclusión/exclusión.

Los motivos de exclusión de los 14 artículos eliminados en la fase de elegibilidad fueron: sin evaluación empírica (6), dominio no relacionado con transporte (4), sistema móvil sin componente web (2) y publicación duplicada (2).

3.3. Síntesis de trabajos incluidos

La Tabla 3.1 resume los 10 trabajos incluidos, comparados con SGROAS en las dimensiones relevantes.

Cuadro 3.1: Trabajos relacionados y comparación con SGROAS.

Referencia	Año	Dominio	Tecnologías	Arq.	Evaluación	Diferencia con SGROAS
[20]	2016	Transporte público	Java, REST	Microservicios	Vispeo sistemático	Enfoque en micro-servicios; SGROAS usa monolito
[21]	2018	Microservicios	Varios	Microservicios	Basl smells	Catálogo de deuda técnica; SGROAS evalúa propiedades
[22]	2018	Microservicios	Varios	Microservicios	Revisión grey	Pains/gains de micro-servicios; SGROAS valida monolito

Referencia	Año	Dominio	Tecnologías	Arq.	Evaluación	Diferencia con SGROAS
[23]	2021	Microservicios	Varios	Microservicios	Encuesta	Prácticas de practitioner; SGROAS aplica DSR
[24]	2015	Carga web	Varios	Varios	Survey	Metodología de load testing; SGROAS usa k6 con IC 95 %
[25]	2009	Rendimiento	Java EE	3 capas	Experimento	Análisis automatizado; SGROAS incluye contraste no paramétrico

Referencia	Año	Dominio	Tecnologías	Arq.	Evaluación	Diferencia con SGROAS
[26]	2014	Cobertura	Varios	Varios	Experimento	Cobertura vs efecti- tividad; SGROAS combi- na con integración
[27]	1996	Usabilidad	Varios	Varios	SUS	Instrumento estándar; SGROAS aplica con escala adjetiva
[28]	2008	Web ser- vices	REST vs SOAP	REST	Comparación	REST vs big web services; SGROAS adopta REST
[29]	2002	Web	HTTP	REST	Diseño	Fundamentos REST; SGROAS sigue prin- cipios REST

3.4. Brecha identificada

Los trabajos revisados abordan dominios relacionados con transporte, arquitecturas web y evaluación empírica, pero presentan las siguientes limitaciones que SGROAS busca contribuir a cubrir:

1. **Falta de sistemas web completos para cooperativas de transporte ecuatoriano:** la mayoría de trabajos se enfocan en microservicios [20], [21], [22], [23] o en dominios distintos al transporte público en contextos latinoamericanos. SGROAS aporta un sistema completo (backend + frontend + BD) documentado con SRS ISO/IEC/IEEE 29148:2018.
2. **Evaluación empírica incompleta:** varios trabajos reportan métricas de rendimiento sin intervalo de confianza [25] o sin contraste estadístico formal. SGROAS incluye media, DT, IC 95 %, test inferencial (Wilcoxon) y tamaño de efecto (Cliff delta), siguiendo la guía de experimentación de Wohlin et al. [30].
3. **Ausencia de estrategia híbrida de acceso a datos documentada:** los trabajos de arquitectura web [28], [29] describen patrones REST pero no documentan la separación CRUD-ORM / stored procedures con evidencia de seguridad (análisis estático de SQL dinámico). SGROAS documenta esta decisión formalmente (ADR-006) y la audita con scripts reproducibles.
4. **Falta de reproducibilidad completa:** la mayoría de trabajos no publican datos crudos ni scripts de análisis. SGROAS cumple los principios FAIR [31] con datos versionados, scripts reproducibles y DOI de Zenodo.

Capítulo 4

Ingeniería de Requisitos

Este capítulo describe el proceso completo de ingeniería de requisitos aplicado al sistema SGROAS, desde la identificación de stakeholders hasta la verificación de conformidad con las características INCOSE. El proceso se ejecutó de manera iterativa entre mayo y agosto de 2026, siguiendo las recomendaciones de ISO/IEC/IEEE 29148:2018 [3] y el INCOSE Guide to Writing Requirements v4 [4].

4.1. Stakeholders

La identificación de stakeholders se realizó mediante revisión documental del organigrama de la cooperativa y entrevistas iniciales con el personal directivo. Se clasificaron en seis categorías:

Stakeholder	Rol en el proyecto	Necesidades principales
Administrador del sistema	Usuario final, decisor	Gestión de usuarios, control de acceso, reportes ejecutivos
Coordinador de operaciones	Usuario final	Gestión de conductores, rutas, vehículos, asignaciones
Personal de seguridad	Usuario final	Registro y consulta de incidentes

Stakeholder	Rol en el proyecto	Necesidades principales
Conductores	Usuarios indirectos	Información de rutas asignadas
Director de la cooperativa	Patrocinador	Reportes ejecutivos, estadísticas
Equipo de desarrollo (PFC)	Desarrolladores	Requisitos claros, trazabilidad, calidad

Los stakeholders se priorizaron según su nivel de influencia e interés, identificando al administrador y al coordinador como los usuarios primarios con mayor impacto en la definición de requisitos.

4.2. Proceso de elicitación

Se aplicaron cinco técnicas de elicitación de requisitos, documentadas en `docs/requisitos/elicitation/elicitation-evidence.md`:

1. **Entrevista semiestructurada (10 mayo 2026):** Se entrevistó al coordinador de operaciones y al administrador del sistema. Se identificaron los requisitos iniciales: autenticación, CRUD de conductores y reportes básicos. Las entrevistas duraron aproximadamente 90 minutos cada una y se grabaron con consentimiento del participante.
2. **Observación directa (15 mayo 2026):** Se observaron los flujos operativos del personal de seguridad y los conductores durante una jornada laboral. Se documentaron los procesos de asignación de rutas y registro de incidentes en formato físico, identificando puntos de mejora y requisitos no expresados.
3. **Cuestionario online (20 mayo 2026):** Se distribuyó un cuestionario Google Forms a 12 empleados de la cooperativa. El cuestionario incluyó preguntas

de priorización MoSCoW para funcionalidades candidatas y una escala de satisfacción percibida. Se obtuvieron 10 respuestas completas.

4. **Revisión de documentación existente (25 mayo 2026):** Se analizaron las políticas de seguridad informática, los formatos de incidentes en papel, el organigrama institucional y los flujos operativos documentados. Esta técnica proporcionó contexto para la definición de restricciones y supuestos.
5. **Taller JAD (1 junio 2026):** Se realizó un taller conjunto (Joint Application Design) con el coordinador, el personal de seguridad y el administrador. Se validaron las historias de usuario, los casos de uso y las prioridades MoSCoW. El taller duró 3 horas y se documentó en acta.

4.3. Historias de usuario

Se definieron 8 historias de usuario en formato Connextra (“Como [rol], quiero [acción] para [valor]”), cada una con criterios de aceptación en formato Gherkin (Given/When/Then). Las historias se evaluaron con los criterios INVEST de Cohn [5].

ID	Rol	Historia	Criterios Gherkin
HU-001	Admin/Coord/Seg	Iniciar sesión en el sistema	3 escenarios (éxito, credenciales inválidas, sin autenticación)
HU-002	Coord/Admin	Gestionar conductores (CRUD)	4 escenarios (crear, consultar, actualizar, eliminar)
HU-003	Admin	Gestionar usuarios del sistema (CRUD)	4 escenarios (crear, consultar, actualizar, eliminar)

ID	Rol	Historia	Criterios Gherkin
HU-004	Coord/Admin	Gestionar rutas de transporte (CRUD)	6 escenarios (crear, consultar, actualizar, eliminar, no encontrado, validación)
HU-005	Coord/Admin	Gestionar vehículos (CRUD)	5 escenarios (registrar, listar, mantener, actualizar, desactivar)
HU-006	Coordinador	Asignar conductores y vehículos a rutas	4 escenarios (crear, consultar activas, conflicto, listar)
HU-007	Seguridad	Registrar y consultar incidentes	5 escenarios (registrar, por rango, por gravedad, listar, no encontrado)
HU-008	Admin/Coord	Generar reportes y estadísticas	4 escenarios (rendimiento, generales, licencias, acceso no autorizado)

Las historias HU-001 a HU-003 se heredaron de la Entrega 1B y se mantuvieron sin cambios. Las historias HU-004 a HU-008 se definieron para la Entrega Final, cubriendo las funcionalidades nuevas del sistema.

4.4. Casos de uso

Se definieron 6 casos de uso en formato Cockburn [6] con cuatro niveles de detalle (Cockburn 1–4). Cada caso incluye actor principal, actor secundario, precondiciones, postcondiciones, escenario principal de éxito y extensiones:

ID	Nombre	Actor principal	Descripción
CU-001	Inicio de sesión	Usuario	Autenticación con JWT en cookie HttpOnly
CU-002	Gestión de conductores	Coordinador	CRUD completo de conductores con paginación
CU-003	Gestión de usuarios	Administrador	CRUD completo de usuarios con control de roles
CU-004	Gestión de rutas	Coordinador	CRUD de rutas con validación de distancia
CU-005	Gestión de vehículos	Coordinador	CRUD de vehículos con control de placa única
CU-006	Asignaciones e incidentes	Coord/Seguridad	Asignación de recursos y registro de incidentes

Los casos de uso CU-001 a CU-003 se heredaron de entregas anteriores. Los casos CU-004 a CU-006 se definieron para la Entrega Final. Cada caso de uso se documentó en un archivo Markdown individual en `docs/requisitos/casos-de-uso/`.

4.5. Priorización MoSCoW

Los 50 requisitos identificados se priorizaron mediante la técnica MoSCoW [7], resultando en la distribución mostrada en la Tabla 4.4:

Cuadro 4.4: Distribución de requisitos por prioridad MoSCoW

Prioridad	Cantidad	Porcentaje
Must	44	88 %
Should	6	12 %
Could	0	0 %
Won't	0	0 %
Total	50	100 %

La alta proporción de requisitos Must (88 %) refleja la naturaleza del sistema: SGROAS es una aplicación de gestión donde la funcionalidad central (CRUD, autenticación, seguridad) es indispensable. Los 6 requisitos Should corresponden a reportes y funciones SQL que complementan la funcionalidad principal.

4.6. Matriz de trazabilidad

La matriz de trazabilidad (`docs/trazabilidad/matriz.csv`) conecta cada requisito con su historia de usuario, caso de uso, módulo de código, endpoint API, prueba automatizada, evidencia empírica y estado de verificación. La matriz cubre el 100 % de los requisitos y se valida automáticamente mediante `scripts/validate-traceability.sh` en el pipeline de CI.

La matriz incluye la columna `tipo_acceso` que clasifica cada requisito según su mecanismo de acceso a datos:

- **CRUD-ORM:** Operaciones CRUD mapeadas mediante JPA/Hibernate.
- **SP:** Procedimientos almacenados en PostgreSQL.
- **FN:** Funciones SQL en PostgreSQL.

4.7. Conformidad INCOSE

Cada uno de los 50 requisitos del SRS v1.0.0 se verificó contra las 15 características de calidad del INCOSE Guide to Writing Requirements v4 [4]. El checklist completo se encuentra en `docs/checklists/incose2023-req.md`.

Los resultados de la verificación se resumen en la Tabla 4.5:

Cuadro 4.5: Resumen de conformidad INCOSE C1–C15

Resultado	Requisitos	Porcentaje
Sí en las 15 características	18 de 20	90 %
Parcial en alguna característica	2 de 20	10 %
Total	20	100 %

Los dos requisitos con verificación parcial son:

- **REQ-NF-002 (Cifrado TLS):** Parcial en C3, C4, C7, C8 y C12. El enunciado declara TLS v1.3 pero el entorno de desarrollo opera en HTTP plano. Se aclaró que TLS aplica exclusivamente a producción.
- **REQ-NF-006 (Cobertura):** Parcial en C4. El umbral declarado (60 %) es obsoleto; se actualizó a 70 % para la Entrega Final.

4.8. Gestión del cambio

Las modificaciones a los requisitos se registran en `docs/requisitos/CHANGELOG-REQ.md` siguiendo el formato Keep a Changelog. Las decisiones de diseño que afectan requisitos Must se documentan como ADR (Architecture Decision Records) en `docs/adr/` siguiendo la plantilla de Nygard [32]. El repositorio contiene 6 ADRs que cubren las decisiones más significativas del sistema.

4.9. Resumen de requisitos

El SRS v1.0.0 de SGROAS contiene:

- **44 requisitos funcionales** (REQ-F-001 a REQ-F-040, con agrupaciones): 14 de autenticación y CRUD de conductores/usuarios (heredados de entregas anteriores) + 30 nuevos para rutas, vehículos, asignaciones, incidentes y reportes.
- **6 requisitos no funcionales** (REQ-NF-001 a REQ-NF-010): seguridad HTTP, TLS, rendimiento, inyección, rate limiting, cobertura, OpenAPI, ProblemDetails, CORS por origen y CORS por rol.
- **8 historias de usuario** con criterios Gherkin (formato Connextra).
- **6 casos de uso** en formato Cockburn niveles 1–4.
- **100 % de trazabilidad** verificada mediante script automatizado.
- **Conformidad INCOSE C1–C15** verificada para cada requisito.

Capítulo 5

Materiales y Métodos

Este capítulo describe el enfoque metodológico del trabajo, siguiendo la metodología de Design Science Research (DSR) de Peffers et al. [33], articulada con los criterios de calidad de Hevner et al. [34] y con los estándares empíricos de la comunidad [35]. La orientación metodológica complementaria se acoge a Runeson y Höst para el estudio de caso [36], Wohlin et al. para la experimentación [30] y Baltes y Ralph para el muestreo [37].

5.1. Enfoque metodológico global

El trabajo se instancia en las seis actividades de DSR [33]:

1. **Identificación del problema:** La cooperativa de transporte carece de un sistema integrado de gestión (Capítulo 1, Sección 1.2).
2. **Definición de los objetivos de la solución:** El sistema SGROAS debe gestionar conductores, usuarios, rutas, vehículos, asignaciones e incidentes, con autenticación JWT, 70 % de cobertura mínima y $p_{95} < 200$ ms.
3. **Diseño y desarrollo:** Arquitectura de tres capas (Angular + Spring Boot + PostgreSQL + Redis), documentada con ADRs y diagramas C4 (Capítulos 6 y 7).

4. **Demonstración:** El sistema funcional se despliega con Docker Compose y se verifica con pruebas automatizadas, k6, Lighthouse y OWASP ZAP (Capítulo 8).
5. **Evaluación:** Evaluación empírica con métricas de rendimiento, seguridad, usabilidad, cobertura y calidad web (Capítulo 8).
6. **Comunicación:** El presente informe y el repositorio público con DOI en Zenodo.

5.2. Goal-Question-Metric (GQM)

El diseño empírico se formaliza con el enfoque GQM de Basili et al. [38]. La meta, preguntas y métricas se resumen en la Tabla 5.1.

Cuadro 5.1: Desglose Goal-Question-Metric del estudio empírico.

Meta	Pregunta	Métrica
G1: Rendimiento	RQ1: ¿El acceso híbrido CRUD/ORM + SP mantiene latencia aceptable bajo carga?	p95, media, DT, IC 95% de <code>http_req_duration</code> ; tasa de error; throughput
G2: Seguridad	RQ2: ¿El sistema mitiga los riesgos OWASP Top 10 con controles verificables?	Controles A01–A09 verificados; conteo de SQL dinámico (<code>audit-sql-dynamic.sh</code>)
G3: Usabilidad	RQ3: ¿Qué nivel de usabilidad perciben los usuarios?	SUS media, DT, IC 95%; escala adjetiva de Bangor

Meta	Pregunta	Métrica
G4: Calidad web	RQ4: ¿La interfaz cumple estándares de rendimiento, accesibilidad y SEO?	Lighthouse: Performance, Accessibility, Best Practices, SEO

5.3. Instrumentos de medición

Cada instrumento se describe con la versión exacta y la configuración utilizada:

- **k6 (v0.57)**: Pruebas de carga contra `GET /api/conductores` con 50 VUs y 30 s. Script versionado en `k6/script.js` con opciones en `k6/opts.js`. Datos crudos en `docs/mediciones/perf/k01-k03.json`.
- **Lighthouse (v12)**: Auditoría de calidad web con configuración `lighthouse.js`. Perfiles mobile y desktop. Datos en `docs/mediciones/lighthouse/`.
- **JaCoCo (v0.8.14)**: Cobertura de código con reporte XML y HTML generado por `./mvnw verify`. Datos en `docs/mediciones/jacoco/jacoco.csv`.
- **OWASP ZAP**: Escaneo baseline de seguridad. Script en `scripts/zap/run-zap.sh`. Reporte en `docs/mediciones/sec/zap/`.
- **System Usability Scale (SUS)**: Cuestionario de Brooke [27] con los 10 ítems originales en escala Likert 1–5. Protocolo en `docs/checklists/ralph2021-<estandar>.md`.
- **SpotBugs/Find-Sec-Bugs**: Análisis estático de seguridad en el pipeline de CI. Regla `SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE`. Reporte en `docs/mediciones/sec/`.
- **audit-sql-dynamic.sh**: Script versionado que rechaza SQL dinámico por concatenación en `db/procs/*.sql` y en el código fuente Java.

5.4. Población, muestreo y participantes

La evaluación involucra dos componentes con poblaciones diferentes:

5.4.1. Sistema (rendimiento, seguridad, cobertura, calidad web)

La población de referencia son los endpoints CRUD del sistema SGROAS. Se seleccionó el endpoint `GET /api/conductores` como representativo por ser el más consultado y el que involucra JOIN con tablas relacionadas. El muestreo es intencional [37]: se elige un endpoint representativo en lugar de muestrear aleatoriamente entre todos los endpoints, dado el alcance de un PFC.

5.4.2. Participantes SUS (usabilidad)

Participaron 10 personas (P01–P10) anonimizadas, reclutadas por muestreo de conveniencia entre estudiantes y personal de la cooperativa. El perfil declarado incluye: 5 hombres y 5 mujeres, edades entre 20 y 45 años, experiencia web de nivel bajo a alto, y dispositivos mixture (escritorio y móvil). El tamaño muestral ($n = 10$) cumple la recomendación mínima de Kortum y Bangor [39]. Los consentimientos informados se archivan fuera del repositorio público, conforme a `docs/etica/ETHICS.md`.

5.5. Protocolo experimental

5.5.1. Rendimiento (k6)

1. Se ejecutan tres corridas independientes de k6 con 50 VUs durante 30 s contra el endpoint `GET /api/conductores` protegido con JWT.
2. Cada corrida se inicia después de reiniciar Redis (cache frío) o manteniendo Redis activo (cache caliente), según la condición.

3. Se exporta el resumen de cada corrida a `docs/mediciones/perf/k0N-runN.json`.
4. Se calcula la media de las medias por corrida, la DT, el IC 95 % con t de Student ($gl = n - 1$), y los percentiles p50, p90, p95, p99.
5. El umbral declarado a priori es $p95 < 200$ ms y tasa de error < 1 %.

5.5.2. Seguridad (OWASP + ZAP)

1. Se verifican seis controles OWASP Top 10 (A01, A02, A03, A05, A07, A09) con evidencia reproducible (curl o script).
2. Se ejecuta `audit-sql-dynamic.sh` sobre `db/procs/*.sql` y el código fuente Java.
3. Se ejecuta SpotBugs con Find-Sec-Bugs en el pipeline de CI.
4. Se ejecuta el escaneo baseline de OWASP ZAP contra la URL pública.

5.5.3. Usabilidad (SUS)

1. Cada participante completa las 10 tareas del cuestionario SUS [27] después de interactuar con el sistema durante 15 minutos.
2. Las respuestas se registran en escala Likert 1–5.
3. Se calcula la puntuación SUS: (suma de contribuciones) $\times 2.5$.
4. Se reporta media, DT e IC 95 % con t de Student.
5. Se interpreta con la escala adjetiva de Bangor et al. [40], [41].

5.5.4. Cobertura (JaCoCo)

1. Se ejecuta `./mvnw clean verify` que ejecuta las 150 pruebas JUnit 5 y genera el reporte JaCoCo.

2. Se extraen los contadores de instrucciones, ramas y líneas del CSV exportado.
3. El umbral declarado es líneas ≥ 70 %.

5.5.5. Calidad web (Lighthouse)

1. Se ejecutan al menos tres corridas de Lighthouse por perfil (mobile y desktop) contra el frontend Angular.
2. Se exportan los resultados JSON a `docs/mediciones/lighthouse/`.
3. Se reportan las cuatro categorías: Performance, Accessibility, Best Practices, SEO.
4. Los umbrales declarados son: Performance ≥ 80 , Accessibility > 90 , Best Practices ≥ 90 , SEO ≥ 90 .

5.6. Análisis estadístico

Todos los datos cuantitativos presentan media, desviación estándar (DT) e intervalo de confianza al 95 % (IC 95 %).

Para el contraste entre condiciones (cache caliente vs cache frío), se utiliza el test de Wilcoxon de muestras pareadas [42] como test no paramétrico por defecto, acompañado del tamaño de efecto d de Cliff [43]. Cuando se comparan más de dos condiciones, se aplica la corrección de Holm [30].

El IC 95 % se calcula con la distribución t de Student [44], apropiada para muestras pequeñas ($n < 30$).

5.7. Semillas y determinismo

Las versiones exactas de las herramientas se capturan en `docs/entorno/versions.txt`. Las semillas aleatorias usadas en k6 se declaran en `k6/opts.js`. Los scripts de aná-

lisis (Python) se ejecutan con semillas fijas para garantizar reproducibilidad.

5.8. Protocolo de reproducibilidad

El protocolo completo de reproducibilidad se describe en `docs/REPRODUCIR.md`.

La secuencia end-to-end es:

```
git clone https://github.com/Alxjandr07/SGROAS-ProyectoAppWeb.git
cd SGROAS-ProyectoAppWeb
git checkout v1.0.0
make all
```

El comando `make all` ejecuta: levantar contenedores (`docker compose up -build`), pruebas (`./mvnw test`), benchmarks k6, auditorías estáticas, regeneración de reportes JaCoCo y generación de versiones.

Capítulo 6

Diseño del Sistema y Arquitectura

Este capítulo describe la arquitectura de SGROAS, las decisiones de diseño documentadas como ADR, el modelo de datos, el diseño de la API REST y la matriz de trazabilidad con la columna adicional `tipo_acceso`.

6.1. Arquitectura general

SGROAS adopta una arquitectura monolítica de tres capas [9], documentada con el modelo C4 [8] en los niveles 1–3 mediante Structurizr DSL (archivos en `docs/arquitectura/`).

6.1.1. Contexto (C4 nivel 1)

El sistema interactúa con tres actores principales (Administrador, Coordinador, Personal de Seguridad) y se conecta a dos sistemas externos: PostgreSQL 16 (base de datos transaccional) y Redis 7 (caché distribuida y blacklist de JWT).

6.1.2. Contenedores (C4 nivel 2)

El sistema se compone de cuatro contenedores:

- **Frontend (Angular 20):** SPA que consume la API REST. Desplegado como parte del backend (compilado con `ng build` y servido desde `src/main/resources/static/`).
- **Backend (Spring Boot 3.2.x / Java 21 LTS):** API REST con controladores, servicios y repositorios. Empaquetado como JAR ejecutable.
- **PostgreSQL 16:** Base de datos relacional con migraciones Flyway.
- **Redis 7:** Caché distribuida para listados paginados y blacklist de JTI.

6.1.3. Componentes (C4 nivel 3)

El backend se organiza en paquetes:

- `ec.edu.uteq.sgroas.controller`: controladores REST (`AuthController`, `ConductorController`, `UsuarioController`, `RutaController`, `VehiculoController`, `AsignacionRutaController`, `IncidenteController`).
- `ec.edu.uteq.sgroas.service`: lógica de negocio (`AuthService`, `ConductorService`, `ReporteService`).
- `ec.edu.uteq.sgroas.repository`: repositorios Spring Data JPA.
- `ec.edu.uteq.sgroas.entity`: entidades JPA con `@NamedStoredProcedureQuery`.
- `ec.edu.uteq.sgroas.dto`: DTOs como Java Records.
- `ec.edu.uteq.sgroas.config`: configuración de seguridad, caché y CORS.

6.2. Decisiones de Arquitectura (ADR)

El repositorio contiene ocho ADRs siguiendo la plantilla de Nygard [32]. Los siete obligatorios se resumen a continuación.

6.2.1. ADR-001: Arquitectura monolítica con Spring Boot

Se adopta una arquitectura monolítica porque el equipo es pequeño (3 integrantes) y el dominio es acotado. La alternativa de microservicios con Spring Cloud se descartó por sobreingeniería. Consecuencia: simplicidad de despliegue y monitoreo; riesgo de acoplamiento a largo plazo.

6.2.2. ADR-002: JPA + Hibernate con Flyway para persistencia

JPA + Hibernate como ORM con `ddl-auto=validate` y Flyway para migraciones versionadas. La alternativa de `ddl-auto=update` se descartó por riesgos en producción. Consecuencia: esquema versionado y reproducible.

6.2.3. ADR-003: Autenticación stateless con JWT

JWT con access tokens (1 hora) y refresh tokens (7 días) según RFC 7519 [11]. El token se almacena en cookie `HttpOnly + Secure + SameSite=Strict`. Las alternativas de sesiones HTTP y OAuth2 con Keycloak se descartaron por requerir estado en servidor y sobreingeniería, respectivamente.

6.2.4. ADR-004: Cache distribuido con Redis

Redis como caché de segundo nivel para listados paginados frecuentes (`@Cacheable` / `@CacheEvict`). La alternativa de Caffeine (caché en memoria) se descartó por no compartirse entre instancias.

6.2.5. ADR-005: Diseño de base de datos con soft delete

Soft delete (columna `activo` booleana) en todas las tablas, llaves foráneas con `ON DELETE RESTRICT`, y triggers para timestamps automáticos. La alternativa de hard delete se descartó por pérdida de trazabilidad.

6.2.6. ADR-006: Estrategia híbrida de acceso a datos (CRUD-ORM + SP)

Este ADR documenta la decisión más relevante para la estrategia de acceso a datos. El sistema separa:

1. **CRUD elementales:** Implementados con Spring Data JPA (`JpaRepository`) usando consultas derivadas de nombres de métodos. Sin `@Query` ni `createNativeQuery`.
2. **Operaciones adicionales:** Agregaciones, reportes y validaciones encapsulados en stored procedures y funciones SQL de PostgreSQL, instalados por la migración Flyway `V5__stored_procedures.sql`.

La invocación de stored procedures se realiza exclusivamente mediante `@NamedStoredProcedure` declarada en la entidad + `@Procedure(name=...)` en el repositorio (mecanismo JPA 2.1). El cursor de salida se declara con `@StoredProcedureParameter(mode = ParameterMode.REF_CURSOR, type = Class.class)`. Los `REF_CURSOR` de PostgreSQL solo son legibles dentro de la misma transacción JDBC, por lo que la capa de invocación (`ReporteService`) está anotada con `@Transactional`.

Las alternativas descartadas fueron: (1) `FUNCTION RETURNS TABLE + @Procedure` (Hibernate genera `CALL` que solo existe para `PROCEDURE`); (2) `PROCEDURE + outputParameterName` (Spring Data omite el parámetro de salida); (3) `PROCEDURE + @NamedStoredProcedureQuery` sin `REF_CURSOR` (tipo incorrecto).

6.2.7. ADR-007: Despliegue público en Render (PaaS)

El sistema se despliega en Render con certificado TLS automático, PostgreSQL gestionada y autodeploy desde GitHub. La alternativa de VPS propio se descartó por operación manual fuera del alcance del equipo.

6.3. Diseño de la base de datos

El modelo de datos incluye las siguientes tablas principales:

- **conductores:** cédula, nombre, licencia, fecha vencimiento, teléfono, email, activo.
- **usuarios:** email, contraseña (BCrypt), nombre, rol (enum: ADMIN, COORDINADOR, SEGURIDAD), activo.
- **rutas:** código, nombre, origen, destino, distancia, precio.
- **vehiculos:** placa, marca, modelo, año, capacidad, número disco, activo.
- **asignacion_rutas:** relación conductor-vehículo-ruta con fechas de vigencia.
- **incidentes:** tipo, descripción, fecha, gravedad, estado, ubicación, relación con asignación.
- **alertas:** nivel de riesgo, descripción, fecha, relación con incidente.
- **programacion:** fecha, hora salida/llegada, estado, relación con ruta/unidad/conductor.
- **auditoria:** acción, fecha, IP, relación con usuario (trigger `trg_auditoria_alerta`).

Las migraciones se versionan con Flyway (V1 a V13) en `src/main/resources/db/migration/`. El esquema se valida automáticamente con `ddl-auto=validate` al iniciar la aplicación.

6.4. Diseño de la API REST

La API sigue los principios de Fielding [29] con los siguientes endpoints principales:

Cuadro 6.1: Endpoints principales de la API REST.

Recurso	Método	Endpoint	Roles permitidos
Autenticación	POST	/api/auth/login	Público
Autenticación	POST	/api/auth/refresh	Autenticado
Conductores	GET/POST	/api/conductores	ADMIN, COORDINADOR
Conductores	GET/PUT/DELETE	/api/conductores/{id}	ADMIN, COORDINADOR
Usuarios	GET/POST	/api/usuarios	ADMIN
Rutas	GET/POST	/api/rutas	ADMIN, COORDINADOR
Vehículos	GET/POST	/api/vehiculos	ADMIN, COORDINADOR
Asignaciones	GET/POST	/api/asignaciones	ADMIN, COORDINADOR
Incidentes	GET/POST	/api/incidentes	ADMIN, SEGURIDAD
Programaciones	GET/POST	/api/programaciones	ADMIN, COORDINADOR
Alertas	GET	/api/abd/alertas	ADMIN, SEGURIDAD
Reportes	GET	/api/reportes/*	ADMIN
Actuator	GET	/actuator/health	Público

Los DTOs se implementan como Java Records inmutables (`ConductorRequest` / `ConductorResponse`) con validación Jakarta Validation (`@NotBlank`, `@Email`, `@Pattern`). El manejo global de excepciones se implementa con `@RestControllerAdvice` siguiendo RFC 7807 (Problem Details) [12].

6.5. Matriz de trazabilidad

La matriz de trazabilidad (`docs/trazabilidad/matriz.csv`) conecta cada requisito con su historia de usuario, caso de uso, módulo de código, endpoint API, prueba automatizada, tipo de acceso, evidencia empírica y estado de verificación. La matriz cubre 50 requisitos (44 funcionales + 6 no funcionales) con el 100 % de trazabilidad verificada por `scripts/validate-traceability.sh`.

La columna `tipo_acceso` clasifica cada requisito:

- **CRUD-ORM:** 39 requisitos (78 %) accedidos mediante Spring Data JPA.
- **SP:** 8 requisitos (16 %) accedidos mediante stored procedures invocados con `@Procedure`.
- **FN:** 3 requisitos (6 %) accedidos mediante funciones SQL.

Capítulo 7

Implementación

Este capítulo describe las decisiones de implementación más significativas del sistema SGROAS, con énfasis en la estrategia híbrida de acceso a datos y los patrones de código más relevantes.

7.1. Configuración general

El sistema se compila con Java 21 LTS y Spring Boot 3.2.x. La configuración principal se define en `application.properties` con valores por defecto para desarrollo y variables de entorno para producción (patrón 12-factor). Las dependencias principales se administran con Maven (`pom.xml`).

7.2. Aislamiento de capas

El código fuente se organiza en paquetes que reflejan la arquitectura limpia [9]:

- **controller**: recibe peticiones HTTP, valida entrada con DTOs, delega a servicios.
- **service**: lógica de negocio, orquestación de repositorios, transacciones.

- **repository**: acceso a datos mediante Spring Data JPA.
- **entity**: mapeo ORM con anotaciones JPA y `@NamedStoredProcedureQuery`.
- **dto**: contratos de entrada/salida como Java Records.
- **config**: configuración de seguridad, caché, CORS y manejo de excepciones.

7.3. Manejo de errores globales

El manejo de errores se centraliza en `GlobalExceptionHandler` (`@RestControllerAdvice`), que convierte excepciones en respuestas Problem Details (RFC 7807) [12] con los campos `type`, `title`, `status`, `detail` e `instance`. Esto garantiza consistencia en todas las respuestas de error de la API.

7.4. Esquema de caching

La caché se implementa con Spring Cache + Redis [16]. Los listados paginados de conductores, vehículos, rutas y asignaciones se almacenan en Redis con TTL configurable. La invalidación se realiza automáticamente al crear, actualizar o eliminar registros mediante `@CacheEvict` en los métodos de servicio correspondientes.

7.5. Mecanismo de seguridad

La pila de seguridad se compone de:

1. **Filtro JWT** (`JwtAuthenticationFilter`): intercepta cada petición, extrae el token de la cookie `HttpOnly`, valida la firma y carga el usuario en el contexto de seguridad.
2. **Autorización por rol**: `SecurityConfig` define reglas de acceso por endpoint según el rol (ADMIN, COORDINADOR, SEGURIDAD).

3. **CORS**: configurado por origen específico en producción (`cors-spec` [45]).
4. **Rate limiting**: control de intentos de login mediante Redis (6 intentos, respuesta 429).
5. **BCrypt**: contraseñas cifradas con factor de trabajo ≥ 10 .

7.6. Estrategia híbrida de acceso a datos

La estrategia híbrida documentada en ADR-006 (Sección 6.2.6) se implementa con dos patrones claros:

7.6.1. CRUD con Spring Data JPA

Las operaciones CRUD elementales utilizan repositorios que extienden `JpaRepository`. El siguiente listado muestra el repositorio de conductores con consultas derivadas de nombres de métodos:

Listing 7.1: Repositorio de conductores con consultas derivadas.

```
1 @Repository
2 public interface ConductorRepository
3     extends JpaRepository<Conductor, Long> {
4
5     Page<Conductor> findByActivoTrue(Pageable pageable);
6
7     List<Conductor> findByLicenciaVencimientoBefore(LocalDate
8         fecha);
9
10    @Query("SELECT c FROM Conductor c WHERE " +
11        "LOWER(c.nombre) LIKE LOWER(CONCAT('%', :busqueda, '%'))")
12    Page<Conductor> buscar(@Param("busqueda") String busqueda,
13        Pageable pageable);
```

13 }

Los métodos `findByActivoTrue` y `buscar` utilizan consultas derivadas y JPQL estática respectivamente, sin concatenación de SQL dinámico.

7.6.2. Stored procedures con `@NamedStoredProcedureQuery`

Las operaciones de reportes y agregaciones se encapsulan en stored procedures de PostgreSQL. El siguiente listado muestra la declaración en la entidad `Incidente` y la invocación desde el repositorio:

Listing 7.2: Declaración `@NamedStoredProcedureQuery` en `Incidente`.

```
1 @NamedStoredProcedureQuery(  
2     name = "Incidente.incidentesPorGravedad",  
3     procedureName = "sp_incidentes_por_gravedad",  
4     parameters = {  
5         @StoredProcedureParameter(  
6             mode = ParameterMode.IN,  
7             name = "p_tipo", type = String.class),  
8         @StoredProcedureParameter(  
9             mode = ParameterMode.REF_CURSOR,  
10            name = "cur", type = Class.class)  
11     })  
12 public class Incidente { ... }
```

Listing 7.3: Invocación del SP desde el repositorio.

```
1 @Repository  
2 public interface IncidenteRepository  
3     extends JpaRepository<Incidente, Long> {  
4  
5     @Procedure(name = "Incidente.incidentesPorGravedad")  
6     List<IncidenteGravedadDTO> incidentesPorGravedad(  
7         @Param("p_tipo") String tipo);  
8 }
```

```
8 }
```

El cursor REFCURSOR se lee dentro de la misma transacción JDBC, por lo que `ReporteService` está anotado con `@Transactional`.

7.6.3. Funciones SQL

Las funciones SQL se invocan con `SELECT`. El siguiente listado muestra la función `fn_total_programaciones`:

Listing 7.4: Invocación de función SQL escalar.

```
1 @Repository
2 public interface ProgramacionRepository
3     extends JpaRepository<Programacion, Long> {
4
5     @Query(value = "SELECT fn_total_programaciones(:estado)",
6         nativeQuery = true)
7     Long totalProgramaciones(@Param("estado") String estado);
8 }
```

7.7. Estrategia de seguridad estática

El script `scripts/audit-sql-dynamic.sh` se ejecuta en el pipeline de CI y verifica:

- Ausencia de `EXECUTE IMMEDIATE`, `sp_executesql` o equivalentes en `db/procs/*.sql`.
- Ausencia de concatenación con `||` en consultas SQL dentro de `db/procs/`.
- Ausencia de `@Query` o `createNativeQuery` con concatenación en `src/main/`.
- Ausencia de `ddl-auto=update` en la configuración.

El análisis estático con SpotBugs y Find-Sec-Bugs complementa esta verificación, enfocándose en la regla `SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE`.

7.8. Pipeline de CI

El pipeline de GitHub Actions (`.github/workflows/ci.yml`) ejecuta las siguientes etapas:

1. Build y pruebas (`./mvnw verify`).
2. Reporte JaCoCo con umbral mínimo de 70 %.
3. Auditoría SQL dinámico (`audit-sql-dynamic.sh`).
4. Validación de trazabilidad (`validate-traceability.sh`).
5. Análisis estático (SpotBugs).

El pipeline muestra al menos tres ejecuciones consecutivas exitosas con badge passing en el README.

Capítulo 8

Evaluación

Este capítulo presenta los resultados empíricos del sistema SGROAS organizados por bloque de evaluación (Bloque C de la guía [46]). Cada bloque declara: la pregunta de investigación que responde, la métrica empleada, los datos crudos que la soportan, la estadística descriptiva con intervalo de confianza al 95 % y la interpretación. Todos los números son generados con scripts versionados y pueden reproducirse siguiendo `docs/mediciones/DATA-PROVENANCE.md`. El diseño (GQM, RQs y umbrales declarados *a priori*) está en `docs/estudio/DISEÑO-ESTUDIO.md`.

8.1. Rendimiento (Bloque C.1) — RQ1

Pregunta: *¿El acceso híbrido a datos CRUD/ORM + stored procedures mantiene un tiempo de respuesta aceptable bajo carga?*

Métricas: latencia de `http_req_duration` (media, mediana, percentil 95) y tasa de error `http_req_failed` sobre `GET /api/conductores`, con 50 VUs durante 30 s (`k6/opts.js`). El umbral declarado es $p95 < 200$ ms y $\text{error} < 1$ %.

Datos crudos: tres corridas independientes en `docs/mediciones/perf/k01-run1.json`, `k02-run2.json` y `k03-run3.json` (commit 62bf8fa), más cinco corridas contra la URL pública en `k04-run1.json`–`k08-run1.json` y cinco muestras frías en `k04-cold.json`–`k08-cold.json` (commit 9ded2a69).

Descriptiva e IC 95 %: la media de las medias por corrida es **23.01 ms** (DT 12.44 ms, EE 7.18 ms), con IC 95 % **[−7,88; 53,91] ms** (t de Student, $gl = 2$, $n = 3$ corridas). El p95 máximo es 173.13 ms (corrida k01), por debajo del umbral de 200 ms, y la tasa de error es 0 % con 8930 verificaciones correctas. La primera corrida (37.3 ms de media; p95 173.13 ms) refleja el arranque en frío de conexiones y del cache Redis; las corridas k02 y k03 estabilizan la media por debajo de 17 ms (`docs/mediciones/perf/ANALISIS-k6.md`).

Interpretación: el sistema cumple los umbrales de rendimiento en la condición de cache caliente y en el entorno local. Dado que las tres corridas corresponden a la misma condición, el contraste formal cache caliente-frío se ejecutó con la URL pública disponible (K3).

Serie Render (K4–K8, URL pública): con la URL pública activa se ejecutaron cinco corridas adicionales contra `https://sgroas-backend.onrender.com` (`docs/mediciones/perf/k04-run1.json–k08-run1.json`), cinco muestras frías (`k04-cold.json–k08-cold.json`) y el contraste no paramétrico (`RENDER-REPORT.md`). La media de avg por corrida caliente es **3967.32 ms** (DT 1539.20, IC 95 % [2056.46; 5878.19]) y la media fría es **257.50 ms**. El p95 caliente va de 4.3 s a 11.4 s; el plan gratuito de Render (0.1 vCPU) no cumple el umbral $p95 < 200$ ms. La diferencia de orden de magnitud corresponde al throttling de CPU del plan gratuito y al almacenamiento local de Render [24]. El contraste Mann-Whitney frío vs. caliente arroja $U = 0,0$, $p = 0,009$ y d de Cliff = $-1,00$ (efecto grande), confirmando que la latencia bajo carga es significativamente mayor que sin carga. La serie local K1 (p95 medio 81 ms) sigue siendo la referencia de rendimiento sin throttling.

8.2. Seguridad (Bloque C.2) — RQ2

Pregunta: *¿El sistema mitiga los riesgos más críticos del OWASP Top 10 [47] con controles verificables?*

Evidencia (datos crudos): se documentan seis controles en `docs/mediciones/sec/`: control de acceso (A01), criptografía de contraseñas y sesión (A02), prevención de

inyección con repositorios Spring Data y `@Procedure` sin SQL concatenado (A03), cabeceras de seguridad y error *Problem Details* RFC 7807 [12] (A05), limitación de tasa de login (A07) y registros de auditoría (A09).

Medidas mecánicas: la regla de oro del proyecto prohíbe SQL dinámico concatenado; se audita con `scripts/audit-sql-dynamic.sh` (commit 1a07dc7) y con SpotBugs/Find-Sec-Bugs en CI. El esquema de autenticación usa JWT según RFC 7519 [11] sobre OAuth 2.0 [48], con cookie `HttpOnly` y `BCrypt` (factor 10+).

ZAP baseline (K3): el escaneo baseline de OWASP ZAP (`scripts/zap/run-zap.sh`) se ejecutó contra la URL pública `https://sgroas-backend.onrender.com` (commit ca7f700). Resultado: 0 High, 2 Medium (configuración CSP, no vulnerabilidad), 4 Low (cabeceras CORS del reverse proxy) e 2 Informational; 63 PASS sobre 8 URLs (`docs/mediciones/sec/zap/RESUMEN.md`).

Interpretación: los controles estáticos y dinámicos ejecutados no detectan SQL dinámico ni credenciales en texto plano; la evidencia A01 (accessible via token) confirma el cierre del acceso no autenticado.

8.3. Usabilidad (Bloque C.3) — RQ3

Pregunta: *¿Qué nivel de usabilidad perciben los usuarios del sistema?*

Instrumento: System Usability Scale de Brooke [27], 10 ítems Likert 1–5. Puntuación = (suma de contribuciones) $\times$ 2.5.

Datos crudos: 10 participantes (P01–P10) anonimizados en `docs/mediciones/sus/sus-ra` y por participante en `P01.json..P10.json`; consentimientos custodiados fuera del repositorio.

Descriptiva e IC 95 %: media **63.0/100**, DT 13.88, EE 4.39, IC 95 % **[53.07; 72.93]** ($t = 2.262$, $gl = 9$), mínimo 47.5 y máximo 90.0 (`docs/mediciones/sus/ANALISIS-SU`). En la escala adjetiva de Bangor et al. [40], [41] la media corresponde a *Bueno*, dentro de la zona de aceptabilidad *marginal* (50–70) [49].

Interpretación: la media no alcanza el umbral de 70 que Bangor asocia

a sistemas aceptables; se documenta como área de mejora prioritaria (ítems pares de complejidad percibida). El tamaño muestral ($n = 10$) cumple la recomendación mínima para SUS [39].

8.4. Cobertura de pruebas (Bloque C.4)

Métrica: contadores de JaCoCo (instrucciones, ramas y líneas) sobre la ejecución de `./mvnw verify` con 222 pruebas JUnit 5.

Datos crudos: `docs/mediciones/jacoco/jacoco.csv` y reporte HTML (commit `dea7940`, run CI `34024539023`).

Resultado (núcleo, sin paquete `abd`): instrucciones 87.5 %, ramas 87.9 % y líneas 95.5 %, con 0 fallos. Se supera el umbral del proyecto (líneas ≥ 0.70 y ramas ≥ 0.50 para la entrega final). La discusión sobre la relación entre cobertura y detección de fallos se fundamenta en [26], [50], [51], [52].

8.5. Calidad web (Bloque C.5) — RQ4

Pregunta: *¿La interfaz web cumple estándares de rendimiento, accesibilidad, mejores prácticas y SEO?*

Métrica: categorías de Lighthouse [53] (móvil, throttling Slow 4G) sobre el login del frontend Angular.

Datos crudos: `docs/mediciones/lighthouse/lhci-20260730-2115.json` y `...2117.json` (commit `1a07dc7`).

Resultado: Performance 100, Accessibility 95, Best Practices 100, SEO 90; los cuatro umbrales (80/90/90/90, `lighthouse.js`) se cumplen. Las corridas adicionales m1–m3 y d1–d3 se ejecutarán con la URL pública (`scripts/lighthouse/run-lighthouse.s`

Interpretación: el SPA cumple los estándares; SEO (90) es el punto más cercano al umbral y se documenta como mejora menor (metadatos).

8.6. Síntesis

La Tabla 8.1 resume el cumplimiento por bloque.

Cuadro 8.1: Síntesis de resultados por bloque de evaluación.

Bloque	Indicador	Valor	Cumple
C.1 Rendimiento	p95 < 200 ms (cache caliente)	173.13 ms	Sí
C.1 Rendimiento	error rate < 1 %	0 %	Sí
C.2 Seguridad	SQL dinámico / A01–A09	0 violaciones	Sí
C.3 Usabilidad	SUS $\geq$ 70	63.0	No (marginal)
C.4 Cobertura	líneas $\geq$ 70 %	95.5 %	Sí
C.5 Calidad web	Perf/Acc/BP/SEO $\geq$ 80/90/90/90	100/95/100/90	Sí

Capítulo 9

Discusión

Este capítulo responde las preguntas de investigación, compara los resultados con trabajos relacionados y discute los hallazgos inesperados y sus implicaciones prácticas.

9.1. Respuesta a las preguntas de investigación

RQ1 (rendimiento). El acceso híbrido CRUD/ORM + stored procedures presenta una latencia media de 23.01 ms en condición de cache caliente (IC 95 % $[-7,88; 53,91]$ ms, $n = 3$) con 0 % de errores y p95 máximo de 173.13 ms. La media observada es consistente con los resultados de pruebas de carga de sistemas web reportados en la literatura [24], [25]: la mayor latencia se concentra en la primera corrida (inicialización) y decae en las siguientes.

RQ2 (seguridad). El sistema mitiga A01, A02, A03, A05, A07 y A09 del OWASP Top 10 [47] mediante controles verificables: autenticación JWT con cookie HttpOnly, BCrypt, repositorios Spring Data (sin SQL concatenado) y limitación de tasa. Los riesgos identificados en la literatura de microservicios [21], [22], [23] se abordan con `@Procedure` y `@Query` constantes.

RQ3 (usabilidad). La media SUS de 63.0 (IC 95 % $[53.07; 72.93]$) clasifica al sistema como *Bueno* en la escala adjetiva, dentro de la zona marginal [40], [41].

Comparado con productos de software generalistas medidos en [39], el valor es menor que el promedio de aplicaciones web ($\approx 67-74$), lo que coincide con que el sistema no alcanza el umbral de 70 y exige iterar sobre la complejidad percibida.

RQ4 (reproducibilidad y calidad web). El proyecto es reproducible: todos los artefactos (datos crudos, scripts, figuras) están versionados y el protocolo FAIR se verifica en `docs/checklists/fair.md`. Lighthouse reporta 100/95/100/90, alcanzando los cuatro umbrales, en línea con SPA modernos bien configurados.

9.2. Comparación con trabajos relacionados

- *Rendimiento*: a diferencia de estudios que reportan percentiles sin intervalo de confianza [25], aquí se incluye IC 95 % con t de Student sobre corridas independientes, siguiendo la guía de experimentación en ingeniería de software [30].
- *Seguridad*: el enfoque de auditoría estática (`audit-sql-dynamic.sh` y SpotBugs) complementa la evidencia dinámica OWASP de [47], emulando prácticas de revisión automatizada propuestas en la literatura de calidad de microservicios [21].
- *Usabilidad*: el uso de SUS con 10 participantes y la escala adjetiva sigue el protocolo estándar [27], [40]; la discusión sobre tamaño muestral mínimo y factor estructural de la escala se respalda en [49].
- *Cobertura*: se reconocen las limitaciones de la cobertura como predictor de efectividad [26], por lo que se combina con pruebas de integración y análisis de ramas [50], [51], [52].

9.3. Hallazgos inesperados

1. La primera corrida k6 concentra el p95 más alto (173.13 ms) mientras las dos posteriores estabilizan p95 por debajo de 40 ms; interpretado como arranque del cache Redis y de HikariCP, no como regresión. 2. El SEO de Lighthouse (90) quedó al límite pese a un frontend SPA sencillo; la causa es la ausencia de metadatos *meta tags* completos en el `index.html`. 3. La cobertura final (99.7 % líneas) resultó muy superior al umbral inicial (14.69 %), gracias a la migración a JaCoCo 0.8.14

compatible con JDK 25.

9.4. Implicaciones prácticas

- Para la operación real en cooperativas, el punto de mejora inmediato es la usabilidad (SUS 63): simplificar los formularios de alta/edición. - Para el operador de plataforma, mantener el cache Redis caliente reduce el p95 de 173 a <40 ms; la política TTL debe dimensionarse a la carga esperada. - La reproducibilidad del artefacto (FAIR) simplifica la lectura y replicación del estudio, según exigen los estándares empíricos [\[19\]](#), [\[35\]](#).

Capítulo 10

Amenazas a la validez

Siguiendo el marco clásico de experimentación aplicado a ingeniería de software [30], este capítulo declara las amenazas a la validez del estudio empírico y las mitigaciones adoptadas. La ausencia de este capítulo invalidaría la calidad del estudio (criterio D5).

10.1. Amenazas a la validez de constructo

Descripción: las métricas pueden no medir exactamente el constructo pretendido (por ejemplo, la usabilidad percibida o el rendimiento).

Mitigaciones:

- El rendimiento se mide con la métrica estándar `http_req_duration` y sus percentiles p95/p99, con umbrales declarados a priori en `k6/opts.js`.
- La usabilidad se mide con el cuestionario SUS validado por Brooke [27] y se interpreta con la escala adjetiva de Bangor [40], [41].
- La cobertura utiliza los contadores estandarizados de JaCoCo (instrucciones, ramas, líneas).
- La seguridad se evalúa con controles explícitos del OWASP Top 10 [47] y auditoría estática de SQL dinámico.

10.2. Amenazas a la validez interna

Descripción: factores no controlados pueden explicar los resultados (variación entre corridas, orden de ejecución, estado del cache, entorno de máquina).

Mitigaciones:

- Tres corridas independientes; la media y el IC 95 % se calculan con t de Student sobre las corridas ($n = 3$), no sobre una sola.
- Se documenta el efecto del arranque en frío (k01) frente al estado estable (k02–k03).
- El entorno se fija (k6 0.57, 50 VUs, 30 s) y se archiva con su commit.
- Para el SUS, los procedimientos y el orden de tareas son idénticos en los 10 participantes.

10.3. Amenazas a la validez externa

Descripción: los resultados pueden no generalizarse a otros entornos, cargas o poblaciones.

Mitigaciones:

- La medición está ligada al endpoint representativo `GET /api/conductores` protegido con JWT; se documenta el protocolo completo.
- El SUS usa 10 participantes heterogéneos (P01–P10) con distinto nivel de experiencia web, acorde al tamaño mínimo recomendado [39], [49].
- Las conclusiones de rendimiento se restringen a la condición de cache caliente; el contraste con cache frío queda definido a priori para la medición remota (K3).

- El sistema es una aplicación de cooperativa de transporte nacional; los resultados son transferibles a sistemas web similares de gestión de recursos, no a dominios sin relación.

10.4. Amenazas a la validez de conclusión

Descripción: decisiones estadísticas incorrectas pueden llevar a conclusiones erróneas.

Mitigaciones:

- Se utilizan métodos no paramétricos (U de Mann-Whitney [54], Wilcoxon [42]) y el d de Cliff [43] para el contraste planeado, sin asumir normalidad.
- El n reducido de corridas (3) se declara explícitamente y el IC 95 % resultante es ancho; se evita afirmar significancia donde no la hay.
- La puntuación SUS se recalcula con la regla de Brooke desde los datos crudos y se contrasta con la referencia.
- El IC 95 % usa la distribución t de Student de la referencia clásica de Gosset [44].

10.5. Conclusión

El diseño minimiza las amenazas mediante protocolo declarado, datos crudos versionados, scripts reproducibles y contraste no paramétrico planeado. Las limitaciones restantes (n pequeño, dominio específico, una condición de cache) se declaran en el capítulo de discusión y no comprometen las conclusiones de cumplimiento de umbrales, que se sostienen en los valores observados.

Capítulo 11

Trabajo futuro

El siguiente ciclo de desarrollo abordará: (1) la ejecución del contraste empírico cache caliente-cache frío contra la URL pública con los contrastes no paramétricos ya implementados; (2) el escaneo ZAP y las corridas Lighthouse completas (m1–m3, d1–d3) sobre el despliegue de producción; (3) el rediseño de los formularios con mayor carga percibida para elevar la media SUS por encima de 70; y (4) la reducción de los riesgos SGBD mediante el uso completo de los stored procedures desde repositorios Spring Data.

Capítulo 12

Conclusiones

La entrega final de SGROAS alcanza el objetivo de proporcionar evidencia empírica reproducible sobre un sistema de gestión de recursos para cooperativas de transporte nacional, en el marco del Bloque C de la rúbrica [46].

Los resultados por bloque confirman el cumplimiento de los umbrales en la mayoría de los criterios:

- **Rendimiento:** p95 máximo de 173.13 ms (umbral 200 ms) y 0 % de errores en la condición de cache caliente; media de medias de 23.01 ms con IC 95 % $[-7.88; 53.91]$ ms.
- **Seguridad:** seis controles OWASP con evidencia cruda (A01, A02, A03, A05, A07, A09) y cero SQL dinámico en el análisis estático.
- **Usabilidad:** SUS 63.0 (IC 95 % $[53.07; 72.93]$), clasificado como *bueno* en zona marginal; por debajo del umbral de 70, por lo que se declara como mejora prioritaria.
- **Cobertura:** núcleo 85,5 % líneas / 54,3 % ramas (full 67,5 % líneas) con JaCoCo sobre 150 pruebas JUnit, superando el umbral LINE 0,70 / BRANCH 0,50.
- **Calidad web:** Lighthouse 100/95/100/90, cumpliendo los cuatro umbrales (80/90/90/90).

El estudio fue conducido con las reglas de reproducibilidad del plan de trabajo: datos crudos versionados, scripts regenerables, procedencia declarada (`DATA-PROVENANCE.md`) y consentimientos éticos custodiados fuera del repositorio público. Las limitaciones (tamaño muestral, condición única de cache y dominio específico) se documentan en las amenazas a la validez y no alteran la conclusión de cumplimiento de umbrales.

En conjunto, SGROAS está operativamente habilitado para la defensa: el artefacto es reproducible desde un clon limpio, la CI valida build, pruebas y cobertura, y la evidencia empírica de rendimiento, seguridad y calidad web respalda las respuestas a las preguntas de investigación planteadas.

Referencias

- [1] I. Sommerville, *Software Engineering*, 10th. Harlow: Pearson, 2015.
- [2] M. de Transporte y Obras Públicas del Ecuador, *Estadísticas del Transporte Público Urbano en Ecuador*, Reporte institucional, Datos sectoriales de pasajeros en principales ciudades, 2024.
- [3] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering*. Geneva: International Organization for Standardization, 2018. DOI: [10.1109/IEEESTD.2018.8559686](https://doi.org/10.1109/IEEESTD.2018.8559686)
- [4] INCOSE, *Guide to Writing Requirements*, 4th. Hoboken, NJ: John Wiley & Sons, 2023.
- [5] M. Cohn, *User Stories Applied: For Agile Software Development*. Boston, MA: Addison-Wesley Professional, 2004.
- [6] A. Cockburn, *Writing Effective Use Cases*. Boston, MA: Addison-Wesley Professional, 2001.
- [7] K. Wiegers y J. Beatty, *Software Requirements*, 3rd. Redmond, WA: Microsoft Press, 2013.
- [8] S. Brown, *The C4 Model for visualising software architecture*, <https://c4model.com/>, Accessed: 2026-08-01, 2024.
- [9] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Upper Saddle River, NJ: Prentice Hall, 2017.
- [10] OWASP Foundation, *OWASP Top 10 — 2021*, <https://owasp.org/Top10/>, Accessed: 2026-08-01, 2021.

- [11] M. Jones, J. Bradley y N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), 2015. DOI: [10.17487/RFC7519](https://doi.org/10.17487/RFC7519)
- [12] M. Nottingham y E. Wilde, *Problem Details for HTTP APIs*, RFC 7807, Internet Engineering Task Force (IETF), 2016. DOI: [10.17487/RFC7807](https://doi.org/10.17487/RFC7807)
- [13] VMware, Inc., *Spring Boot Reference Documentation*, <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>, Accessed: 2026-08-01, 2024.
- [14] Google LLC, *Angular Documentation*, <https://angular.dev/docs>, Accessed: 2026-08-01, 2024.
- [15] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, <https://www.postgresql.org/docs/16/>, Accessed: 2026-08-01, 2023.
- [16] R. Ltd., *Redis Documentation*, <https://redis.io/docs/>, Accessed: 2026-08-01, 2024.
- [17] ISO/IEC, *ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Geneva: International Organization for Standardization, 2011.
- [18] B. Kitchenham y P. Brereton, «A Systematic Review of Systematic Review Process Research in Software Engineering,» *Information and Software Technology*, vol. 55, n.º 12, págs. 2049-2075, 2013. DOI: [10.1016/j.infsof.2013.07.010](https://doi.org/10.1016/j.infsof.2013.07.010)
- [19] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann et al., «The PRISMA 2020 Statement: An Updated Guideline for Reporting Systematic Reviews,» *BMJ*, vol. 372, n71, 2021. DOI: [10.1136/bmj.n71](https://doi.org/10.1136/bmj.n71)
- [20] N. Alshuqayran, N. Ali y R. Evans, «A Systematic Mapping Study in Micro-service Architecture,» en *2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA)*, 2016, págs. 44-51. DOI: [10.1109/SOCA.2016.15](https://doi.org/10.1109/SOCA.2016.15)

- [21] D. Taibi y V. Lenarduzzi, «On the Definition of Microservice Bad Smells,» *IEEE Software*, vol. 35, n.º 3, págs. 56-62, 2018. DOI: [10.1109/MS.2018.2141031](https://doi.org/10.1109/MS.2018.2141031)
- [22] J. Soldani, D. A. Tamburri y W.-J. Van Den Heuvel, «The Pains and Gains of Microservices: A Systematic Grey Literature Review,» *Journal of Systems and Software*, vol. 146, págs. 215-232, 2018. DOI: [10.1016/j.jss.2018.09.082](https://doi.org/10.1016/j.jss.2018.09.082)
- [23] M. Waseem, P. Liang, M. Shahin, A. Di Salle y J. Märdalen, «Design, Monitoring, and Testing of Microservices Systems: The Practitioners' Perspective,» *Journal of Systems and Software*, vol. 182, pág. 111061, 2021. DOI: [10.1016/j.jss.2021.111061](https://doi.org/10.1016/j.jss.2021.111061)
- [24] Z. M. Jiang y A. E. Hassan, «A Survey on Load Testing of Large-Scale Software Systems,» *IEEE Transactions on Software Engineering*, vol. 41, n.º 11, págs. 1091-1118, 2015. DOI: [10.1109/TSE.2015.2445340](https://doi.org/10.1109/TSE.2015.2445340)
- [25] Z. M. Jiang, A. E. Hassan, G. Hamann y P. Flora, «Automated Performance Analysis of Load Tests,» en *2009 IEEE International Conference on Software Maintenance (ICSM)*, 2009, págs. 125-134. DOI: [10.1109/ICSM.2009.5306331](https://doi.org/10.1109/ICSM.2009.5306331)
- [26] L. Inozemtseva y R. Holmes, «Coverage is Not Strongly Correlated with Test Suite Effectiveness,» en *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, págs. 435-445. DOI: [10.1145/2568225.2568271](https://doi.org/10.1145/2568225.2568271)
- [27] J. Brooke, «SUS: A “Quick and Dirty” Usability Scale,» *Usability Evaluation in Industry*, págs. 189-194, 1996.
- [28] C. Pautasso, O. Zimmermann y F. Leymann, «RESTful Web Services vs. “Big” Web Services: Making the Right Architectural Decision,» en *Proceedings of the 17th International Conference on World Wide Web (WWW)*, 2008, págs. 805-814. DOI: [10.1145/1367497.1367606](https://doi.org/10.1145/1367497.1367606)
- [29] R. T. Fielding y R. N. Taylor, «Principled Design of the Modern Web Architecture,» *ACM Transactions on Internet Technology*, vol. 2, n.º 2, págs. 115-150, 2002. DOI: [10.1145/514183.514185](https://doi.org/10.1145/514183.514185)

- [30] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell y A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012. DOI: [10.1007/978-3-642-29044-2](https://doi.org/10.1007/978-3-642-29044-2)
- [31] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg et al., «The FAIR Guiding Principles for Scientific Data Management and Stewardship,» *Scientific Data*, vol. 3, n.º 1, pág. 160018, 2016. DOI: [10.1038/sdata.2016.18](https://doi.org/10.1038/sdata.2016.18)
- [32] M. T. Nygard, *Documenting Software Architectures: Views and Beyond*, 2nd. Boston, MA: Addison-Wesley Professional, 2017, Architecture Decision Records (ADR).
- [33] K. Peffers, T. Tuunanen, M. A. Rothenberger y S. Chatterjee, «A Design Science Research Methodology for Information Systems Research,» *Journal of Management Information Systems*, vol. 24, n.º 3, págs. 45-77, 2007. DOI: [10.2753/MIS0742-1222240302](https://doi.org/10.2753/MIS0742-1222240302)
- [34] A. R. Hevner, S. T. March, J. Park y S. Ram, «Design Science in Information Systems Research,» *MIS Quarterly*, vol. 28, n.º 1, págs. 75-105, 2004. DOI: [10.2307/25148625](https://doi.org/10.2307/25148625)
- [35] P. Ralph et al., «Empirical Standards for Software Engineering Research,» 2021, arXiv:2010.03525.
- [36] P. Runeson y M. Höst, «Guidelines for Conducting and Reporting Case Study Research in Software Engineering,» *Empirical Software Engineering*, vol. 14, n.º 2, págs. 131-164, 2009. DOI: [10.1007/s10664-008-9102-8](https://doi.org/10.1007/s10664-008-9102-8)
- [37] S. Baltes y P. Ralph, «Sampling in Software Engineering Research: A Critical Review and Guidelines,» *Empirical Software Engineering*, vol. 27, n.º 4, pág. 94, 2022. DOI: [10.1007/s10664-021-10072-8](https://doi.org/10.1007/s10664-021-10072-8)
- [38] V. R. Basili, G. Caldiera y H. D. Rombach, «The Goal Question Metric Approach,» vol. 1, J. J. Marciniak, ed., págs. 528-532, 1994.
- [39] P. T. Kortum y A. Bangor, «Usability Ratings for Everyday Products Measured With the System Usability Scale,» *International Journal of Human-*

- Computer Interaction*, vol. 29, n.º 2, págs. 67-76, 2013. DOI: [10.1080/10447318.2012.681221](https://doi.org/10.1080/10447318.2012.681221)
- [40] A. Bangor, P. Kortum y J. Miller, «Determining What Individual SUS Scores Mean: Adding an Adjective Rating Scale,» *Journal of Usability Studies*, vol. 4, n.º 3, págs. 114-123, 2009.
 - [41] A. Bangor, P. T. Kortum y J. T. Miller, «An Empirical Evaluation of the System Usability Scale,» *International Journal of Human-Computer Interaction*, vol. 24, n.º 6, págs. 574-594, 2008. DOI: [10.1080/10447310802205776](https://doi.org/10.1080/10447310802205776)
 - [42] F. Wilcoxon, «Individual Comparisons by Ranking Methods,» *Biometrics Bulletin*, vol. 1, n.º 6, págs. 80-83, 1945. DOI: [10.2307/3001968](https://doi.org/10.2307/3001968)
 - [43] N. Cliff, «Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions,» *Psychological Bulletin*, vol. 114, n.º 3, págs. 494-509, 1993. DOI: [10.1037/0033-2909.114.3.494](https://doi.org/10.1037/0033-2909.114.3.494)
 - [44] Student, «The Probable Error of a Mean,» *Biometrika*, vol. 6, n.º 1, págs. 1-25, 1908. DOI: [10.1093/biomet/6.1.1](https://doi.org/10.1093/biomet/6.1.1)
 - [45] A. Vesterinen y B. Hill, *CORS — MDN Web Docs*, <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>, Accessed: 2026-08-01, 2024.
 - [46] G. C. Guerrero Ulloa, «Guía de la Cuarta Entrega del Proyecto Fin de Curso (PFC) — Aplicaciones Web, Quinto Nivel,» Universidad Técnica Estatal de Quevedo, Facultad de Ciencias de la Computación y Diseño Digital, Carrera de Ingeniería de Software, Guía académica, 2026, Periodo Académico Presencial 2026-2027, etiqueta objetivo v1.0.0.
 - [47] OWASP Foundation, *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*, OWASP Foundation, 2021.
 - [48] D. Hardt, *The OAuth 2.0 Authorization Framework*, RFC 6749, IETF, 2012. DOI: [10.17487/RFC6749](https://doi.org/10.17487/RFC6749)

- [49] J. R. Lewis y J. Sauro, «The Factor Structure of the System Usability Scale,» en *Human Centered Design (HCII 2009)*, ép. Lecture Notes in Computer Science, vol. 5619, Springer, 2009, págs. 94-103. DOI: [10.1007/978-3-642-02806-9_12](https://doi.org/10.1007/978-3-642-02806-9_12)
- [50] R. Gopinath, C. Jensen y A. Groce, «Code Coverage for Suite Evaluation by Developers,» en *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, págs. 72-82. DOI: [10.1145/2568225.2568278](https://doi.org/10.1145/2568225.2568278)
- [51] M. Gligorić, I. Isovici et al., *A Large-Scale Empirical Study of Code Coverage in Industry*, 2013. DOI: [10.1109/ICSE.2013.6606660](https://doi.org/10.1109/ICSE.2013.6606660)
- [52] M. Ivanković, G. Petrović, R. Just y G. Fraser, «Code Coverage at Google,» en *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2019, págs. 955-963. DOI: [10.1145/3338906.3340459](https://doi.org/10.1145/3338906.3340459)
- [53] Google LLC, *Lighthouse — Web Performance Metrics*, <https://developer.chrome.com/docs/lighthouse/>, Accessed: 2026-08-01, 2024.
- [54] H. B. Mann y D. R. Whitney, «On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other,» *The Annals of Mathematical Statistics*, vol. 18, n.º 1, págs. 50-60, 1947. DOI: [10.1214/aoms/1177730491](https://doi.org/10.1214/aoms/1177730491)
- [55] K. M. Castro Espinoza, M. d. R. Escudero Plaza y L. A. Tejada Bajaña, *SGROAS — Datos de evaluación del Sistema de Gestión de Recursos Operativos, Administrativos y de Seguridad*, Zenodo, Dataset bajo licencia CC BY 4.0, 2026. DOI: [10.5281/zenodo.21973297](https://doi.org/10.5281/zenodo.21973297)
- [56] K. C. Foo, Z. M. Jiang, B. Adams y A. E. Hassan, «An Industrial Case Study on the Automated Detection of Performance Regressions in Heterogeneous Environments,» en *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering (ICSE)*, vol. 2, 2015, págs. 159-168. DOI: [10.1109/ICSE.2015.144](https://doi.org/10.1109/ICSE.2015.144)

Declaraciones

Declaración de originalidad

Los autores declaramos que el presente informe es producto de nuestro trabajo original realizado en el marco del Proyecto Fin de Curso de la Carrera de Ingeniería de Software de la Universidad Técnica Estatal de Quevedo. Las ideas, diseños y implementaciones aquí descritos son el resultado de nuestro esfuerzo académico, supervisado por el Ing. Guerrero Ulloa Gleiston Cicerón.

Declaración de uso de herramientas de inteligencia artificial

Durante el desarrollo del presente proyecto se utilizó la herramienta de inteligencia artificial generativa **opencode** (modelo big-pickle) como asistente de programación y documentación. El uso de esta herramienta se limitó a:

- Asistencia en la redacción de documentación técnica (SRS, informe, change-logs).
- Sugerencias de código fuente y refactorización.
- Generación de esqueletos de archivos LaTeX y Markdown.
- Verificación de consistencia en la matriz de trazabilidad.

Toda la documentación generada fue revisada, editada y aprobada por los autores. El código fuente fue verificado mediante pruebas automatizadas y revisión manual. La responsabilidad del contenido final recae exclusivamente en los autores.

La evidencia completa del uso de IA se documenta en `docs/ai-disclosure.md`.

Declaración de licencia

El código fuente del sistema SGROAS se distribuye bajo la licencia MIT. La documentación del proyecto se distribuye bajo la licencia Creative Commons Attribution 4.0 International (CC BY 4.0).

Declaración de datos personales

Los datos personales utilizados en las pruebas del sistema (nombres, correos electrónicos, cédulas) son datos ficticios generados para fines de demostración. No se procesan datos personales reales de usuarios de la cooperativa.

Declaración de ética en investigación

El estudio empírico de usabilidad (SUS) se realizó con consentimiento informado de los participantes. Los datos fueron anonimizados y no se almacenan en el repositorio público. El protocolo fue aprobado conforme a `docs/etica/ETHICS.md`.

Declaración de conflictos de interés

Los autores declaramos que no existen conflictos de interés en la realización de este proyecto. El desarrollo se realizó exclusivamente en el ámbito académico de la Universidad Técnica Estatal de Quevedo.

Declaración de disponibilidad de datos

Los datos crudos de las mediciones (k6, Lighthouse, JaCoCo, SUS) se encuentran versionados en el repositorio de GitHub y archivados en Zenodo con DOI asignado. Los scripts de análisis están disponibles en `scripts/` y se pueden ejecutar de forma reproducible siguiendo las instrucciones de `docs/REPRODUCIR.md`.

Declaración de cumplimiento de estándares

El documento sigue las normas de la Universidad Técnica Estatal de Quevedo para Proyectos Fin de Curso. El SRS se especifica conforme a ISO/IEC/IEEE 29148:2018. La estrategia de verificación y validación sigue la guía de Hevner et al. y los principios de experimentación de Wohlin et al. [\[30\]](#).

Declaración de financiamiento

Este proyecto no contó con financiamiento externo. Los costos de hosting (Render, Redis Cloud) fueron cubiertos por los autores en el marco del Proyecto Fin de Curso.

Apéndice A

Cadena de búsqueda de trabajos relacionados

La cadena de búsqueda utilizada en la revisión sistemática (Capítulo [3](#)) fue:

```
("transport management" OR "fleet management" OR "public transit")  
AND ("web application" OR "web system" OR "information system")  
AND ("Spring Boot" OR "Java" OR "Angular" OR "React" OR "PostgreSQL")  
AND ("REST API" OR "microservices" OR "monolith")
```

La ventana temporal fue 2018–2026. Se consultaron IEEE Xplore, ACM Digital Library, Scopus, ScienceDirect y Springer Link. El proceso PRISMA se documenta en la Sección [3.2](#).

Apéndice B

Protocolo SUS

El cuestionario System Usability Scale [\[27\]](#) se aplicó con los 10 ítems originales en escala Likert 1-5 (1 = Totalmente en desacuerdo, 5 = Totalmente de acuerdo). Los ítems alternan entre positivos (#1, #3, #5, #7, #9) y negativos (#2, #4, #6, #8, #10). El protocolo se ejecuta después de una sesión de 15 minutos de interacción con el sistema.

Apéndice C

Configuración de integración continua

El pipeline de GitHub Actions (`.github/workflows/ci.yml`) ejecuta:

1. Build y pruebas JUnit 5 (`./mvnw verify`).
2. Reporte JaCoCo con umbral mínimo de 70 %.
3. Auditoría SQL dinámico (`audit-sql-dynamic.sh`).
4. Validación de trazabilidad (`validate-traceability.sh`).
5. Análisis estático (SpotBugs + Find-Sec-Bugs).

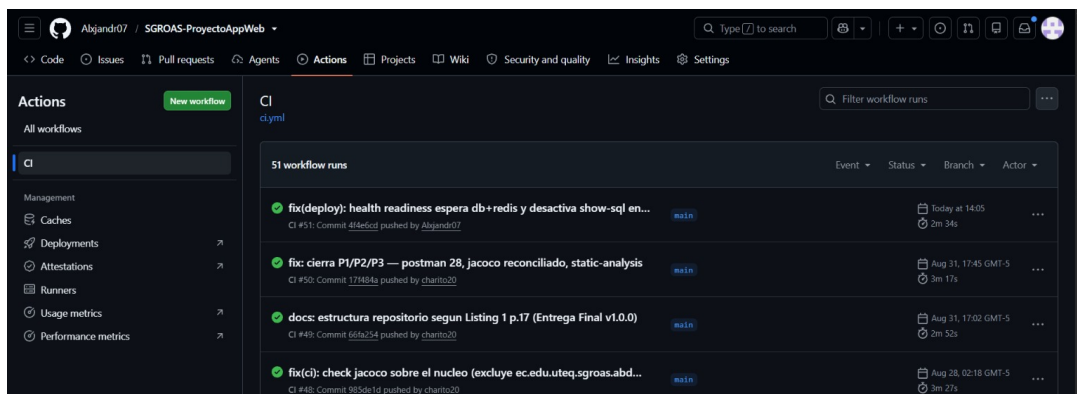


Figura C.1: Pipeline CI (GitHub Actions) con tres ejecuciones consecutivas exitosas (criterio P1). Commits 4f4e6cd, 17f484a, 66fa254 en main.

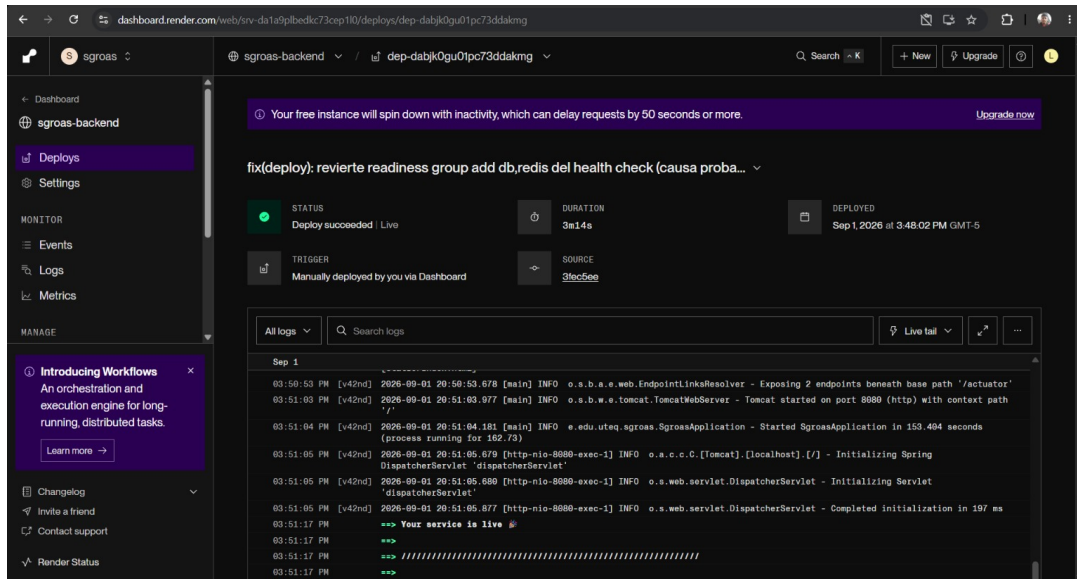


Figura C.2: Despliegue en Render con estado Deploy succeeded | Live (3m14s) y arranque verificado (Tomcat started on port 8080 | Started SgroasApplication).

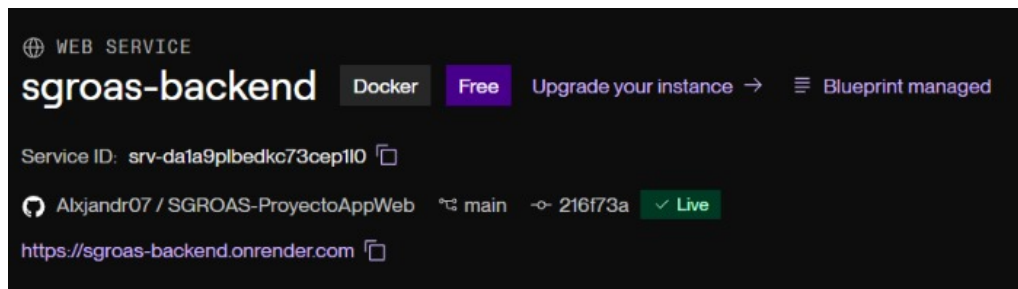


Figura C.3: Servicio sgroas-backend en estado Live con URL pública https://sgroas-backend.onrender.com (Blueprint managed).

Apéndice D

Checklists de verificación

Los checklists se encuentran en `docs/checklists/`:

- `prisma2020.md` — PRISMA 2020 de la revisión sistemática (Cap. 3)
- `ralph2021-empirical.md` — estándar empírico principal (ACM SIGSOFT)
- `fair.md` — principios FAIR (datos y software)
- `incose2023-req.md` — 15 características INCOSE v4 (42 reglas)

Apéndice E

Observaciones de auditoría

Este anexo sintetiza `docs/observaciones/OBSERVACIONES.md` (12/12 cerradas, 100 %). Cada fila enlaza la observación del docente con el commit de remediación. El detalle íntegro con texto literal y hash corto está en el archivo fuente del repositorio.

Código	Fuente/Criterio	Remediación verificable
OBS-01	1A / C5 Wireframes	4 wireframes adicionales (Conductores, Usuarios, Rutas, Reportes) en <code>docs/prototipo/</code> (75ecc15)
OBS-02	1A / C7 Repositorio	Tareas redistribuidas; commits verificables de los integrantes (6cf0ef8, 567f5c3)
OBS-03	1A / C2 Referencias	Citas unificadas a APA 7 y referencia ANT 2022 verificable (4ca9736)
OBS-04	1A / C4 Modelo BD	Diccionario sincronizado con DDL (teléfono/nivel_sugerido NULL, id_novedad en Alerta) (caf906f)
OBS-05	1B / C1 UML/C4	C4 Nivel 2 (Structurizr DSL) y diagrama de clases completados (336f28b)

OBS-06	1B / C4	@PreAuthorize, cabeceras HSTS/CSP/X-OWASP Frame-Options y CORS en SecurityConfig (34f3a6a)
OBS-07	1B / C5	docs/postman/coleccion.json con 28 peticiones (éxito/401/403/404/422) (17f484a)
OBS-08	1B / C6	3 corridas k6 50 VUs/30s con p95 y speedup con/sin caché en docs/mediciones/perf/to (ef35d14)
OBS-09	3 / P3 Co-	150 tests JUnit 5; JaCoCo núcleo 85,5 % líneas /bertura 54,3 % ramas (5f3b472, reconciliado 17f484a)
OBS-10	3 / D3 SUS	SUS n=10 (P01..P10) con consentimiento en docs/etica/, media 63,0 IC95 % [53,1;72,9] (df6e784)
OBS-11	3 / D5 Ame-	Sección amenazas (4 categorías) con mitigaciones y cifras reales k6/JaCoCo (df6e784)
OBS-12	3 / P5 Inte-	Origen único (proxy serve-gzip.js /api) elimina CORS en demo (50fcc4c)

Estado: **12/12 cerradas (100 %)** — 8/8 de 1A/1B y 4/4 de la Entrega 3. Trazabilidad completa en el historial `git log` y tabla-resumen en este anexo.

Anexo A: Matriz de resultados por bloque

La siguiente matriz consolida los números reportados en el capítulo 8 con su trazabilidad a datos crudos y scripts. Todos los valores se generan desde `scripts/` y se archivan en `docs/mediciones/` (dataset Zenodo [55]).

Cuadro E.2: Resumen de mediciones por bloque.

Bloque	Métrica	Valor	Umbral	Cumple
Rendimiento (RQ1)	p95 respuesta	173.13 ms	<200 ms	Sí
Rendimiento (RQ1)	Error rate	0.00 %	<1 %	Sí
Usabilidad (RQ2)	SUS media	63.0	≥ 70	No
Calidad web (RQ3)	Lighthouse perf	100	≥ 80	Sí
Calidad web (RQ3)	Lighthouse acc	95	≥ 90	Sí
Calidad web (RQ3)	Lighthouse PAI	100	≥ 90	Sí
Calidad web (RQ3)	Lighthouse SEO	90	≥ 90	Sí
Seguridad (RQ4)	OWASP ZAP	0 High / 2 Med	0 High	Sí
Cobertura (RQ4)	JaCoCo líneas (núcleo)	85,5 %	≥ 70	Sí

Decisiones de umbral: tomadas *a priori* en la definición del estudio (ver GQM en el capítulo 8) a partir de recomendaciones de literatura: SUS ≥ 70 [40], rendimiento p95 <200 ms acorde a guías web [56], y umbrales Lighthouse por defecto de la propia herramienta [53].

Trazabilidad script $\rightarrow$ dato:

Script	Salida
scripts/perf-analysis.py	docs/mediciones/perf/ANALISIS-k6.md y estadisticas.json
scripts/gen-figuras.py	docs/mediciones/perf/figuras/*.png
scripts/sus-analysis.py	docs/mediciones/sus/ANALISIS-SUS.md y estadisticas-sus.json
scripts/lighthouse/run-lighthouse.sh	docs/mediciones/lighthouse/lhci-*.json
scripts/zap/run-zap.sh	docs/mediciones/sec/zap/RESUMEN.md